Department of Electronics and

Electrical Communications Engineering

Faculty of Engineering - Cairo University

Advanced Processor Architecture

Project

BN	Sec	ID	Name
48	1	9230322	حازم محمد أحمد الششتاوى
8	2	9230359	خالد نبيل ابرهيم الهنداوى
11	2	9230365	رأفت هيثم رأفت عبد الهادي الشيخ
13	2	9230378	روبير تامر رؤوف حنا
23	2	9230427	سامح محمد سامح محمد
25	2	9230434	سعيد ماهر سعيد عطاالله

Submitted to:

Dr. Hossam Fahmi & Eng. Hassan El-Menier

Table of Contents

Table of Figures

0.Introduction

In this report, we will design and implement 8-bit pipelined processor 1-port von Neumann memory architecture through 5-stages using Verilog. Each Pipeline stage will have its functional block with their inputs, and their control signals are handled by MUXs and logic gates with a selection line, selected from the control unit.

The report is organized as sections of Pipeline stages, Hazard Detection and Handling Unit and top module. Each stage will be explained in detail, in its section, showing its functional block design, testbench and the results, while the MUXs and the logic gates will be explained in the top module section.

0.1 Processor specs:

- RISC-like instruction set architecture of 1-byte or 2-byte instructions.
- Four registers of 1-byte size in the general-purpose register file where R_3 is used as SP.
- The memory of von Neumann architecture of size 256 bytes divided by addresses as:
    - Interrupt part: $0 \rightarrow 19$
    - Instruction part: $20 \rightarrow 99$
    - Data part: $100 \rightarrow 199$
    - Stack part: $200 \rightarrow 255$
- The system is always async read data while the write is always sync.
- The reset is async active high control signal.
- The branch is assumed is not taken if taken it will flush the pipeline until the target and condition is known.

0.2 The Pipeline Stages:

The pipeline divides each instruction into five stages:

I. Fetch

II. Decode

III. Execute

IV. Memory

V. Write back

Each stage will have the same clock period, and each instruction will pass through the 5 stages without jumping or anything.

0.3 The Processor Architecture:

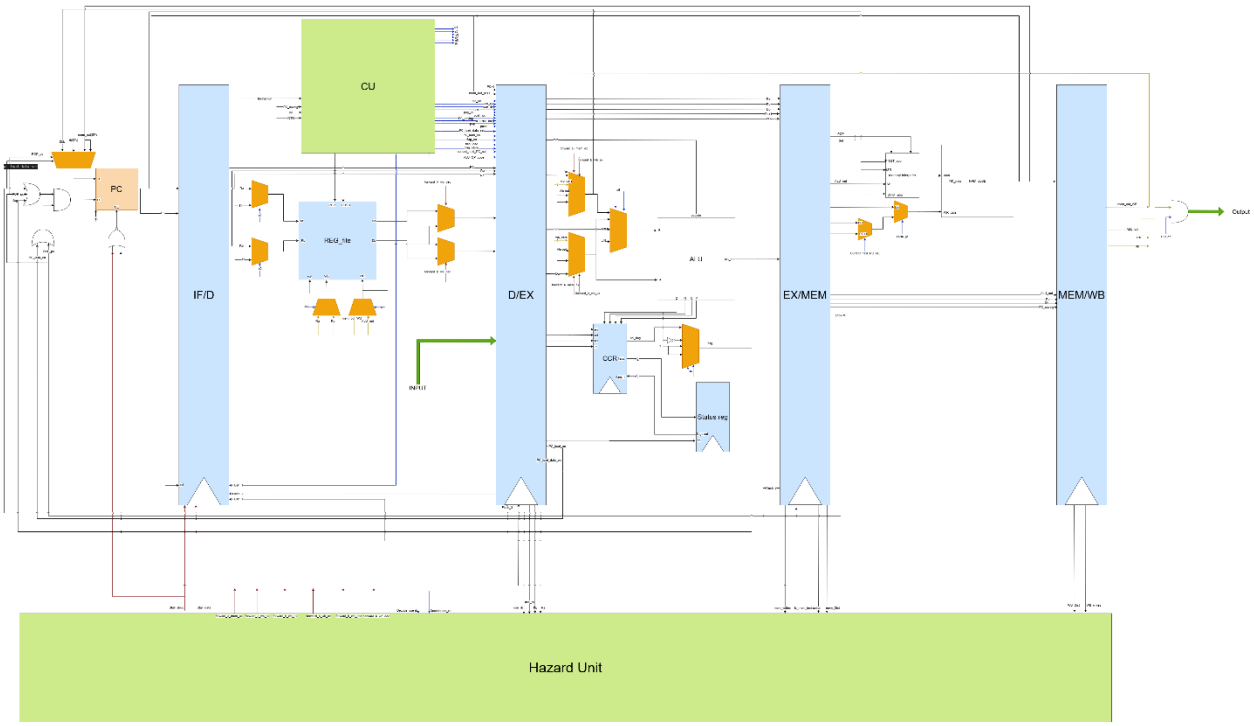

Figure 1: Processor architecture

0.4 FSM Control unit

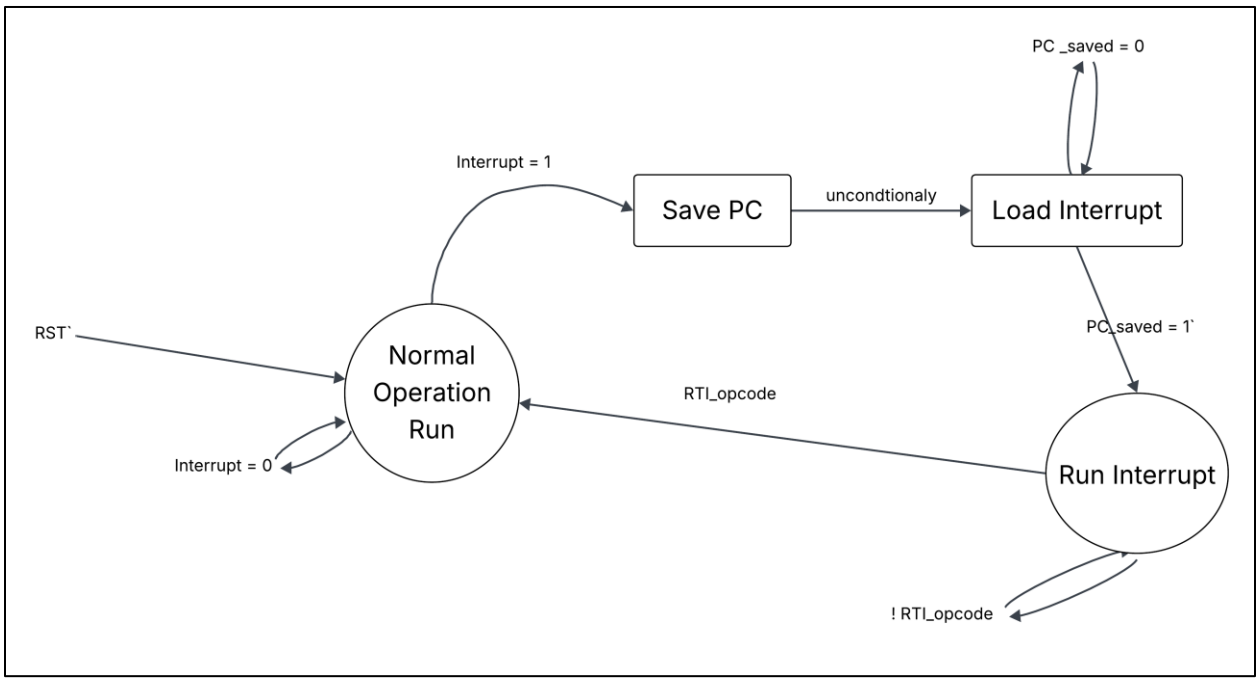

Figure 2: FSM of CU

1. Fetch Stage

It is the first stage of the instruction cycle in a processor. Its function is to retrieve the next instruction from instruction part in the memory using the address stored in the Program Counter (PC) as it sent the PC value on the address bus to the memory interpreter to get the instruction form the memory and loads it into the pipeline register (IF/D) for the next stage. At the end of the stage, the PC is updated (typically incremented or loaded by a branch/jump target address), so it points to the next instruction to be fetched. The functional blocks of this stage are the PC and the memory (will be shown in the memory stage section).

1.1 Program Counter

It is required to either increment so it fetch the next instruction in program or load a value corresponding to the instruction we will jump to. So, it has 2 control signals one is the load enable which allows the PC to load the value given in its input load port and the other is counter enable which enable the PC to increment if any is not high at the positive edge of the clock the PC will keep its value which is useful to do stall if it is required. As the memory address port is 8 bits, it is designed to be 8 bits register that loads or increment the value. It has a reset control signal which is triggered when the program starts to clear the old value of the PC. The reset is async high active control signal.

1.1.1 Program Counter Testbench

The testbench is done to test firstly the reset operation so we made the reset signal high and expect to see the output be low. Then we enabled the load and put a value in load port the output is expected to be the value of the load in the positive edge, this done and the enable of counter which increment the value is low. Then tested the increment so the output expected to be the old value added to it 1, the test done by making the load enable to be low and counter enable to be high. Then tested to make the 2 enable signals to be low to test if it will keep the value which express that PC can do stalls. Then tested the load again with different value in the load port to make sure it can load will working. Then we tested the reset again to make sure it can reset the PC while working.

1.1.2 Program Counter Results

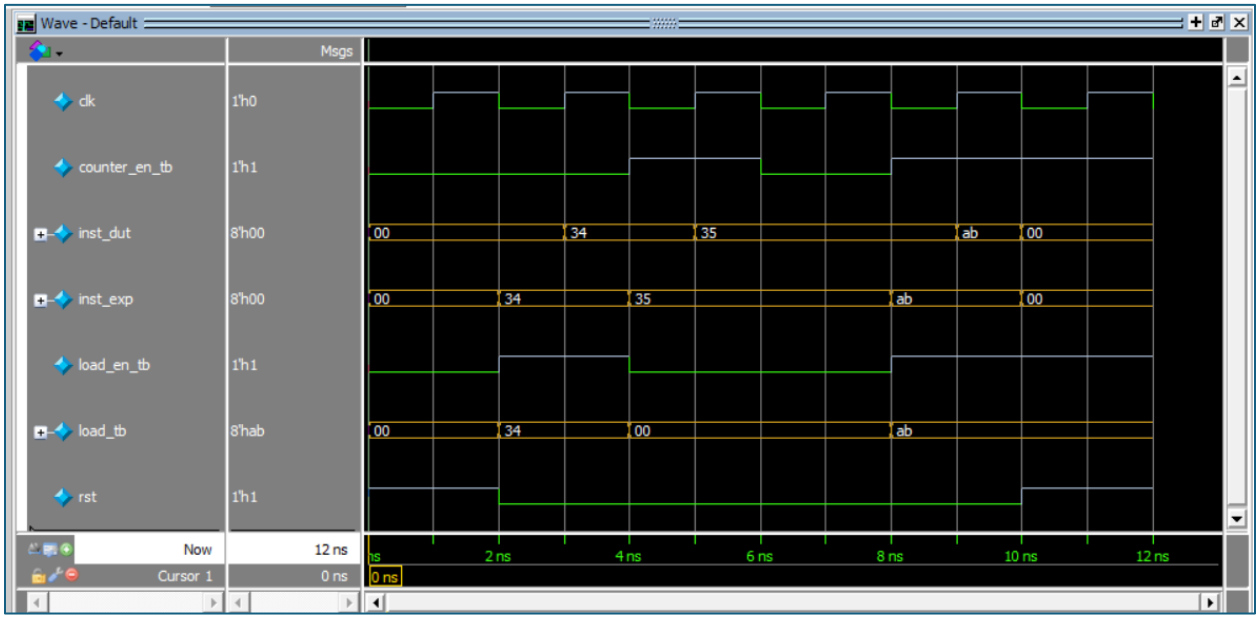

Figure 3: the waveform of the testbench of PC

We can see the waveform show 6 cycles with tests we need:

- 1st cycle is the reset which makes the output to be zero
- 2nd cycle is the load happens which makes the output to be load value
- 3rd cycle is the increment which makes the output to be incremented
- 4th cycle is the stall which makes the output to be the same without change
- 5th cycle is the load happens which makes the output to be load value
- 6th cycle is the reset happens which makes the output to be zero

The output of each cycle is what as expected so we can say that the test has passed and the PC designed as required

2. Decode Stage

The decode stage interprets the fetched instruction and prepares the processor for execution. During this stage, the Control Unit (CU) analyzes the instruction's opcode and fields to determine the required operation and to generate the appropriate control signals for subsequent pipeline stages. In parallel, the instruction's register specifiers are used to access the register file, allowing the required source registers to be read and their contents forwarded to the execution stage. The functional blocks of this stage are register file and CU.

2.1 Register File

It is 4 registers (R0, R1, R2, R3) of 8-bit size designed to have reset signal to make the register file clear with no data except R3 reset to 255 (0xFF) because it is the stack pointer, and control signal for writing to enable writing on positive edge of the clock in it during the Write Back stage and other signal for pop and push which make the R3 "SP" to increment or decrement to be the new value of the pointer to the stack (push has the higher priority). It also takes one 2-bit write address to determine which register to write on and two read addresses of 2-bits to determine each data bus (ra, rb) gets the value of which register. All operations are synchronous except read make sure that a value is on the output bus of the registers file in the same cycle and asynchronous active high reset.

2.1.1 Register File Testbench

A self-checking forced testbench to verify that the Register file module run correctly, at first we check that the reset is working properly for each register, we assert the reset = 1 and the other input is = 0 and choose R0 and R1 to be written on the 2 busses (ra, rb) respectively and compare the output with the expected output value of = 0, then choose R2 and R3 to be written on the 2 busses (ra, rb) respectively and compare the output with the expected value which is equal to zero for ra bus and rb bus = 255. Then testing write, push and pop then read operations, for write operations, assert the reset to be = 0 and the write enable = 1 and make the write data with value assume it = 8'h52 and write in on the one the buses, and expected the same value from the same bus, for push operation, it is expected that the value that is assigned to the R3 to be decremented by 1, for pop operation, , it is expected that the value that is assigned to the R3 to be incremented by 1.

2.1.2 Register File Results

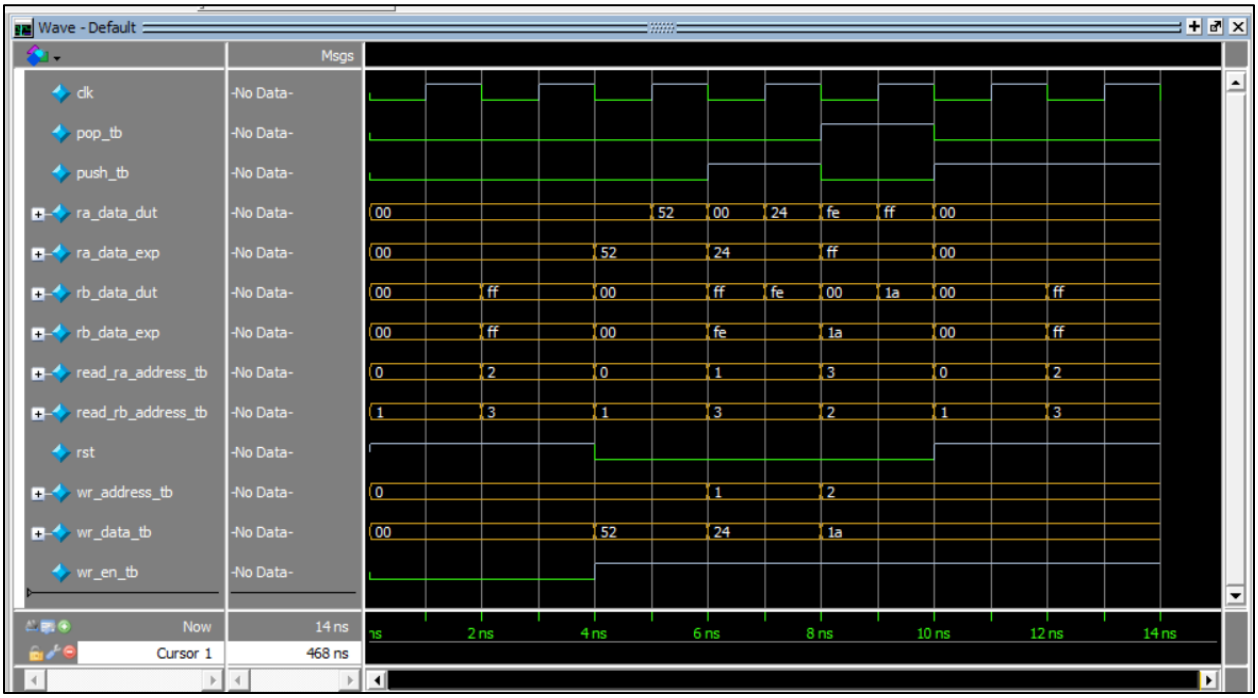

Figure 4: the waveform of the RF testbench

- 1st cycle is resetting then reading R0 and R1 from ra and rb are low.
- 2nd cycle is resetting then reading R2 and R3 from ra is low and rb is FF.
- 3rd cycle is writing a value for R0 then reading R0 and R1 and the output is that value and 0 from ra and rb.
- 4th cycle is written R1 with a value than push then read R1 and R3 and the output is that value and the value R3 is decremented from ra and rb.
- 5th cycle is written R2 a value then pop then read R3 and R2 and the output is that value and the value R3 is incremented from ra and rb.
- 6th cycle is resetting then reading R0 and R1 from ra and rb are low.
- 7th cycle is resetting then reading R2 and R3 from ra is low and rb is FF

The output of each cycle is what as expected so we can say that the test has passed and the RF designed as required

2.2 Control Unit

It has 2 operation modes one for the normal code and one for the interrupt operation. In the normal operation, the control unit selects the control signals for each stage and passes them through the pipeline to be flown with the instruction and select in time of the stage. As the von Neumann architecture is used so handling the fetch stage and memory stage not used at same time is required as we use one port of data. So, CU if the instruction is not using memory is sent

signal to the memory interpreter to make it know that the instruction won't use the memory to make fetch easy otherwise it gives an alert to the hazard unit to see that there is structure hazard may occur. The system predicts any jump to be not taken so when taken the CU should handle the miss, so it flushes the pipeline with number of cycles different for each jump instruction type:

- 2-byte operation flushes 1 cycle
- Any jump except the return flushes 2 cycles as the condition known at the execute stage from the flags
- Return function flushes 3 cycles as the target known in memory stage from the stack value

In the interrupt operation, the CU gets interrupt signal which tell that the ISR should be done so the instruction that is in the CU while interrupt is happened will continue to flow and the next PC gotten from the fetch stage the CU will perform on it a push instruction to save it in the stack and it will save the registers in the CCR in Status register to be gotten back after finishing the interrupt service routine. The cycle after the push the CU sent instruction of load ISR to the PC to be ready to perform the ISR and wait for a signal called PC_savingwhich tells that the PC saved in the stack and can perform the ISR without getting the stop point to be lost. The CU performs the ISR like normal, but it makes the memory interpreter to know that the instructions are for ISR to get them correctly from the right part in the memory. At the end of ISR, there is a return function to make the processor return from where it stopped. The CU perform a normal return function but with loading in CCR the values of the status register. Then the CU returns to normal operation mode.

The control unit is tested with the whole processor when trying to see if each instruction is performed so there is no specified testbench for it.

3. Execute Stage

In this stage the arithmetic and the logic operation are done. It has main three blocks the ALU, CCR the register which saves flags and status register which save the flags when interrupt to be re-loaded in the CCR after finishing the ISR. Each block is designed and then modeled in one block called EX_stage to wrap them in one block and make the implementation in the top module easier. The functional blocks are ALU, CCR, status register and Ex stage wrapper block.

3.1 Arithmetic Logic Unit (ALU)

It is designed to perform different arithmetic and logic operations according to the opcode values gotten from the CU. So, it is designed to take 2 inputs signed A and B and also take the opcode to decide which operation and Cin which will be the carry flag to perform the rotate instructions. It gives as output the result of the operation in signal called "out" and 4 signals represents the flags: zero "Z", negative "N", overflow "V", and carry "C". It also gives update signals to tell the CCR is the opcode given change in the CCR or not. The ALU is combinational, so it gives the output once the inputs are given to it.

Operation	Opcode	Discerption
Addition	0000	Performs signed addition A + B. Updates Z, N, C, V
Subtraction	0001	Performs signed subtraction A − B. Updates Z, N, C, V
AND	0010	Bitwise AND between A and B. Updates Z, N
OR	0011	Bitwise OR between A and B. Updates Z, N
Rotate Left through Carry (RLC)	0100	Rotates B left with Cin entering LSB. Updates C
Rotate Right through Carry (RRC)	0101	Rotates B right with Cin entering MSB. Updates C
Set Carry	0110	Sets Carry flag to 1, output cleared. Updates C
Clear Carry	0111	Clears Carry flag, output cleared. Updates C
NOT B	1000	Bitwise inversion of B. Updates Z, N
NEG B	1001	Two's complement negation of B. Updates Z, N
Increment B	1010	Increments B by 1. Updates Z, N, C, V
Decrement B	1011	Decrements B by 1. Updates Z, N, C, V

Bypass B	1100	Passes B directly to output. No flags updated
Bypass A	1101	Passes A directly to output. No flags updated

3.1.1 Arithmetic Logic Unit (ALU) Testbench

The testbench is not self-checking. It just gives some value for the inputs and from the waveform we decide if it works fine or not. The testbench is made to give 5 different value for the inputs (A, B and Cin) then perform all the possible operations to them be making the opcode changes to all possible values.

3.1.2 Arithmetic Logic Unit (ALU) Results

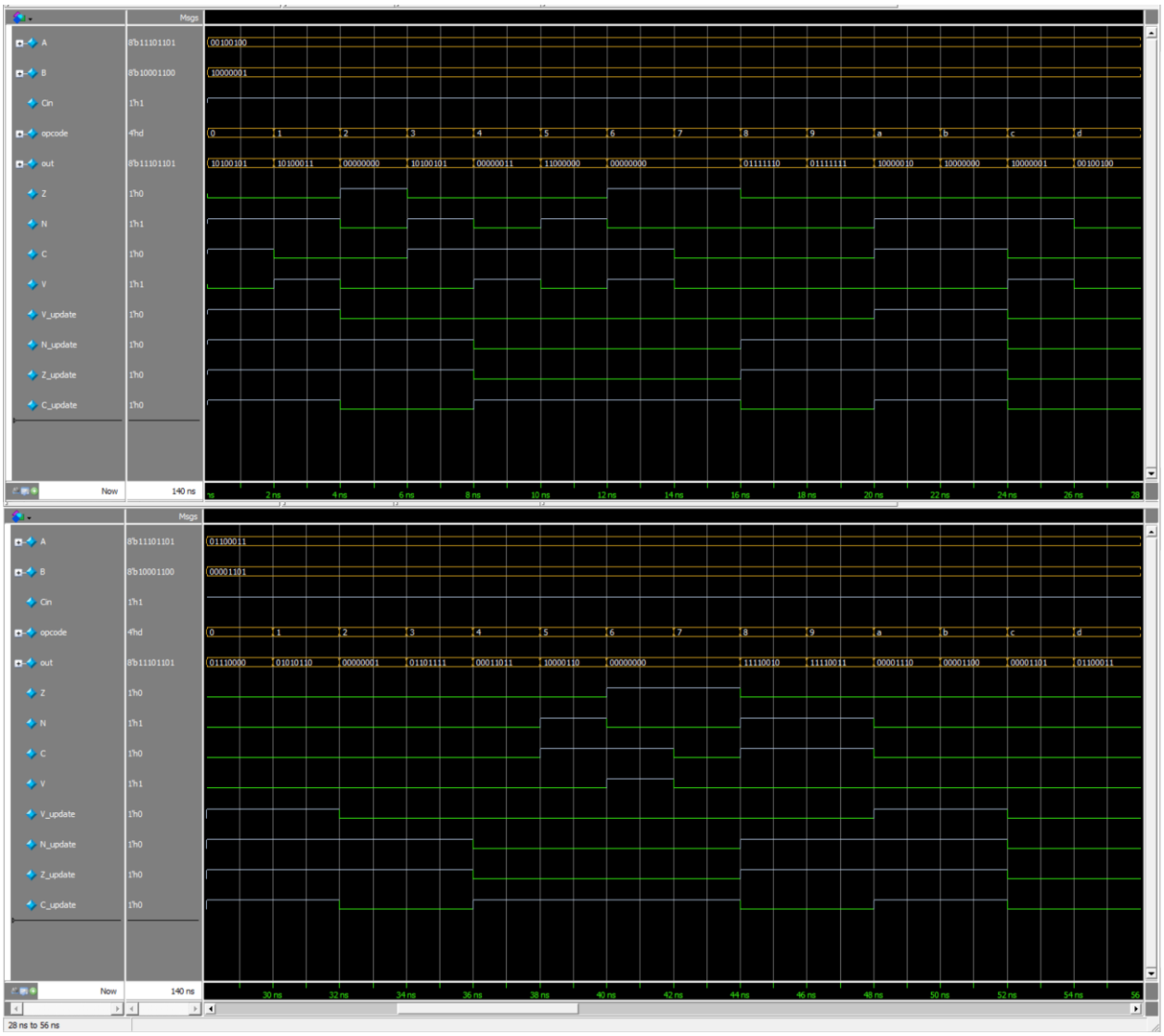

Figure 5: the waveform of the ALU outputs for 1st ~ 2nd inputs values

Figure 6: the waveform of the ALU outputs for 3rd ~ 5th inputs values

We can see that the updated flags values are raised at the required operations and the output of each operation is correct.

3.2 Condition Code Register (CCR)

It is a register of 4 bits to store the flags (CCR[0]: Z, CCR[1]: N, CCR[2]: C, and CCR[3]: V) according to the operation “some operations need to update some flags and others not”. So, CCR has an enable for each bit to know store the value or not and they are coming from ALU. When CU needs to read a flag for some operations or not it gives 2-bit selection signal and the CCR gives the flag according to address async, while it writes them sync. It has a reset signal to clear the register which is async high active control signal. It also has an enable signal for overall CCR which to load the flags if wanted or not. When an interruption occurs and finish and need to restore the old flags saved to be restored so we need to have a load control signal called “status_rsestore” and input to have the load values which is called “status_reg”.

3.2.1 Condition Code Register (CCR) Testbench

The testbench is not self-checking. It just gives some value for the inputs and from the waveform we decide if it works fine or not. It test the reset first then makes 30 cycles with random values for the input it is expected when the restore flags signal is high to restore the signals in the input of the load port when it is low and overall enable is high the CCR flags are updated at the positive edge of clock according to the update flags. The given selection signals give the corresponding value of the CCR address, and the output “flag” should be immediately change.

3.2.2 Condition Code Register (CCR) Results

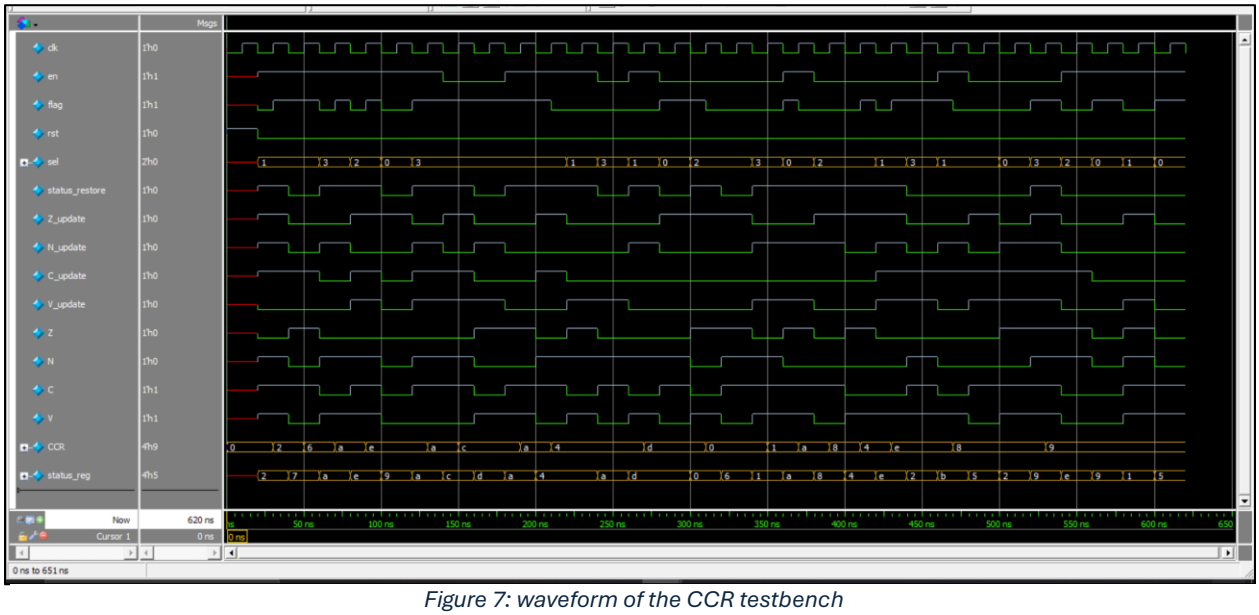

Figure 7: waveform of the CCR testbench

The behavior of the CCR is as expected in each cycle and it is updated at the positive edge of the clock.

3.3 Status Register

It is designed to store the current flags when interrupt hits to get back the values again to continue the code from what stopped. So, it has enable signal to enable storing coming from the CU and it is sync and inputs come from CCR.

3.3.1 Status Register Testbench

It is self-checking testbench. Firstly, tested the reset and expect to have the register to be low. Then enabled the load and randomized the input during 5 periods it is expected for each cycle that the register to have the new input value. Then closed the enable and run for 5 cycles it is expected to keep the old value without change.

3.3.3 Status Register Result

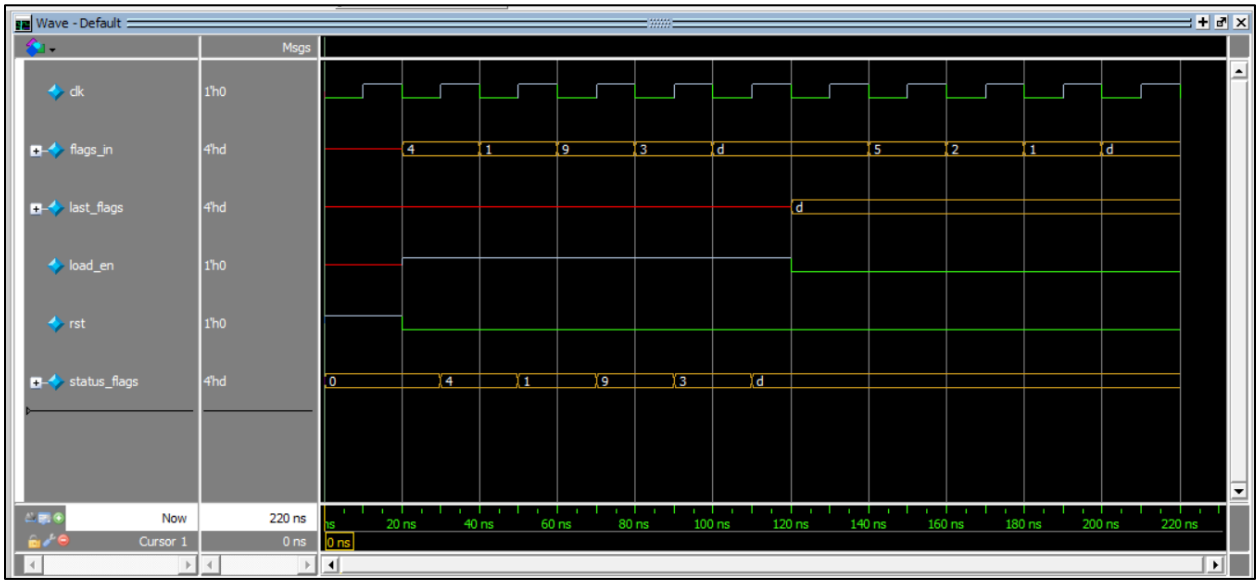

Figure 8: waveform of the status register testbench

- 1st cycle is the reset which makes the output to be zero.
- 2nd ~ 6th cycle is the load happening which makes the output to be the loaded value.
- 7th ~ 11th cycle has load to be low which makes the output to be the same without change.

3.4 Ex Stage block

This is module to connect the ALU, CCR and status register together giving for the top module input signals for the opcode, operands, load flags enable to save the flags in the status register, store flags enable to re-store the old flags in CCR, CCR enable which enable the the CCR. It also gives output as flag the selected from CCR by the CU, zero flag to be used directly in some instructions and the output of the operation from the ALU. It also has clock as input to synchronize the CCR and status register and a reset signal to reset them which is async active high control signal. It is tested with the whole processor when trying to see if each instruction is performed so there is no specified testbench for it.

3.4.1 The block diagram of Ex Stage

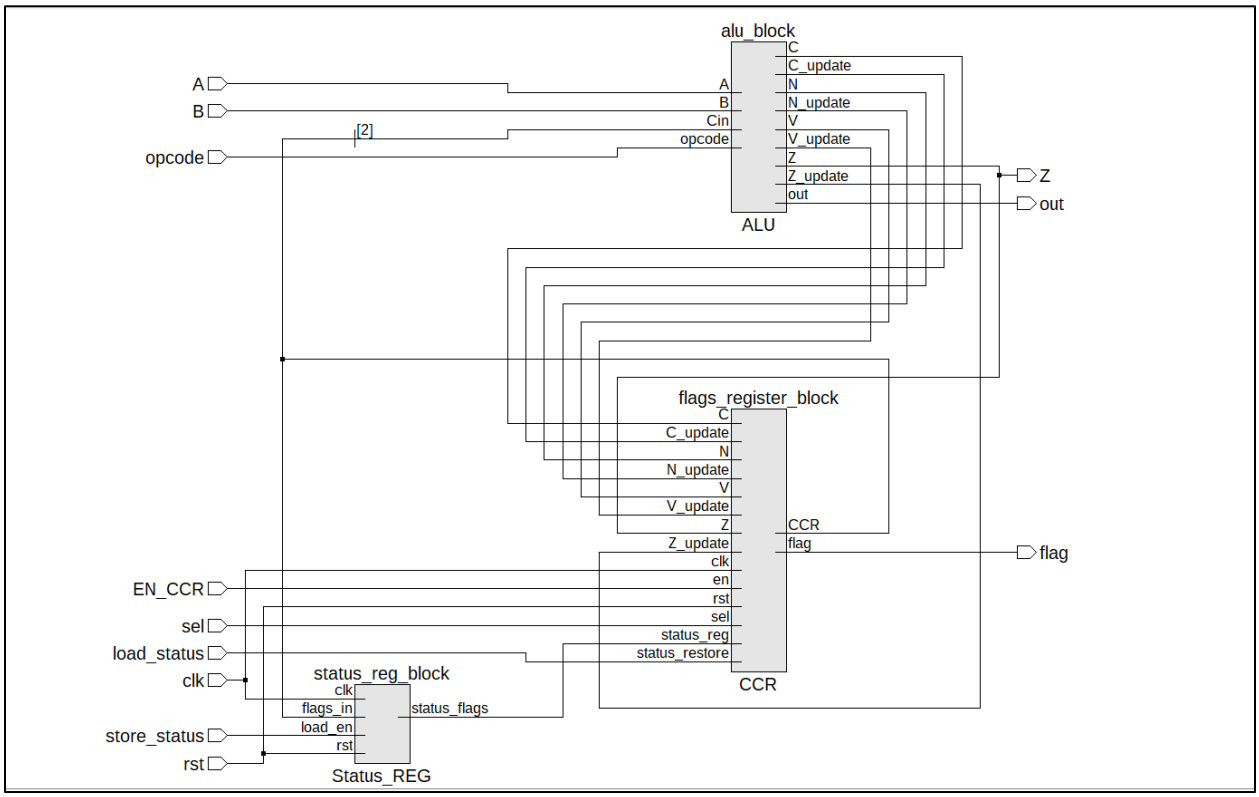

Figure 9: blocks diagram of the EX-stage to be used as one module

4.Memory Stage

In this stage, we use memory in instructions like load/store, but as we use a von Neumann architecture which uses one memory, we divided it into 4 parts for: interrupt (0 ~ 19), instructions (20 ~ 99), data (100 ~ 199) and stack (200 ~255). So, we designed an interpreter which handles an offset to the input address to get the required one as if instruction zero is wanted it adds to the address zero 20 and get the data in address 20 also in others. As it is one port memory, we handled the data structure hazard that fetch stage, and memory used in memory stage which happened at the same time of the fetch by making stall with the help of CU and hazard unit. The CU gives the interpreter a signal if the instruction won't use the memory and it will pass on it without any read or write operation in it. The memory is not used only in the memory stage it is used in fetch as the PC gives the order of the instruction needed to the interpreter and it gives the address equivalent to it to the memory which put on data bus the instruction. Then the interpreter and the memory file are wrapped to be implemented in the top module easier. The functional blocks are interpreter, memory file and memory wrapper block.

4.1 Memory Interpreter

Since we are using von Neumann architecture there is a single memory for both instructions and data, so the function of this module is to set virtual bounders for each type and process the input address to generate the correct address for memory. There are 4 types of data

- Stack pointer defined from 255 to 200.
- Data defined from 199 to 10).
- Instructions defined from 99 to 20.
- Interrupt defined from 19 to 0.

This module defines a constant offset for each type as mentioned above except for the stack pointer as it's already provided with the correct address. The interpreter receives 4 addresses for each type and based on a selection line from the control unit, then pass the correct one to the memory after adding the offset. In case of SP pointer we have two cases push, pop so there is a signal pop from the control unit tell us which operation will be done, in case of push (pop = 0) we need to put data in the address without doing anything as the SP points to the first empty place, but in case of pop we need first to add 1 to the SP so that we point to the data not the empty place.

4.2 Memory File "RAM"

One port memory of size 256 byte each entry of width 8 bits with synchronous write through write enable signal from CU and asynchronous read. It takes an address from the interpreter and if has write enable high it writes in this address if not it doesn't and always give on data bus the data in that address.

4.3 Memory Wrapper

In this, we connected the interpreter with the memory file giving input signals like the reset and the 4 different type of addresses and the selection lines to choose what address with which offset is given to the memory. It gives output the data in the address given on the data bus.

4.3.1 Memory Wrapper Testbench

The testbench tests the memory wrapper using directed read/write tests that ensure correct functional behavior under different access scenarios. It drives the clock and control signals, writes known data patterns to specific addresses, and then reads them back to verify that the data is written correctly and can be accessed by confirming that the stored and retrieved values match. It also tests control signal functionality (such as write enable and read enable) to ensure that memory contents change only during valid write operations and remain stable otherwise. Address sequencing is exercised to confirm correct addressing and isolation between memory locations, while reset behavior (if present) is checked to ensure the memory initializes to a known state.

4.3.2 Memory Wrapper Results

Test behavior:

```
#  STACK OPERATIONS (PUSH/POP)
#
# --- Push Test Case 1 ---
# 40000 UWrite                 255             0xaa
#
# --- Push Test Case 2 ---
# 60000 Write                  254             0xbb
#
#
# === INSTRUCTION MEMORY OPERATIONS ===
#
# --- Instruction Test Case 1 ---
# 70000 WRITE_INST         20 0x12      0x12
# 80000 READ_INST          20           0x12
#
# --- Instruction Test Case 2 ---
# 90000 WRITE_INST         30 0x34      0x34
# 100000 READ_INST         30           0x34
#
#
# === DATA MEMORY OPERATIONS ===
#
# --- Data Test Case 1 ---
# 110000 WRITE_DATA        100 0xcc     0xcc
# 120000 READ_DATA         100          0xcc
#
# --- Data Test Case 2 ---
# 130000 WRITE_DATA        125 0xdd     0xdd
# 140000 READ_DATA         125          0xdd
#
#
# === INTERRUPT VECTOR TABLE OPERATIONS ===
#
# --- Interrupt Test Case 1 ---
# 150000 WRITE_INT          0 0xee      0xee
# 160000 READ_INT           0           0xee
#
# --- Interrupt Test Case 2 ---
# 170000 WRITE_INT          5 0xff      0xff
# 180000 READ_INT           5           0xff
```

Figure 10: test behavior and the data written and read from the memory

The result and compare with the expected:

```
#
# === VERIFICATION - Reading Back All Values ===
# Stack[254] = 0xbb (Expected: 0xBB)
# Stack[255] = 0xaa (Expected: 0xAA)
# Instruction[20] = 0x12 (Expected: 0x12)
# Instruction[30] = 0x34 (Expected: 0x34)
# Data[100] = 0xcc (Expected: 0xCC)
# Data[125] = 0xdd (Expected: 0xDD)
# Interrupt[0] = 0xee (Expected: 0xEE)
# Interrupt[5] = 0xff (Expected: 0xFF)
# Test Complete!
```

Figure 11: the result and comparing with the expected

The output of is as what is expected so we can say that the test has passed and the memory wrapper which has the interpreter and memory file designed as required.

4.3.3 The block diagram of memory wrapper

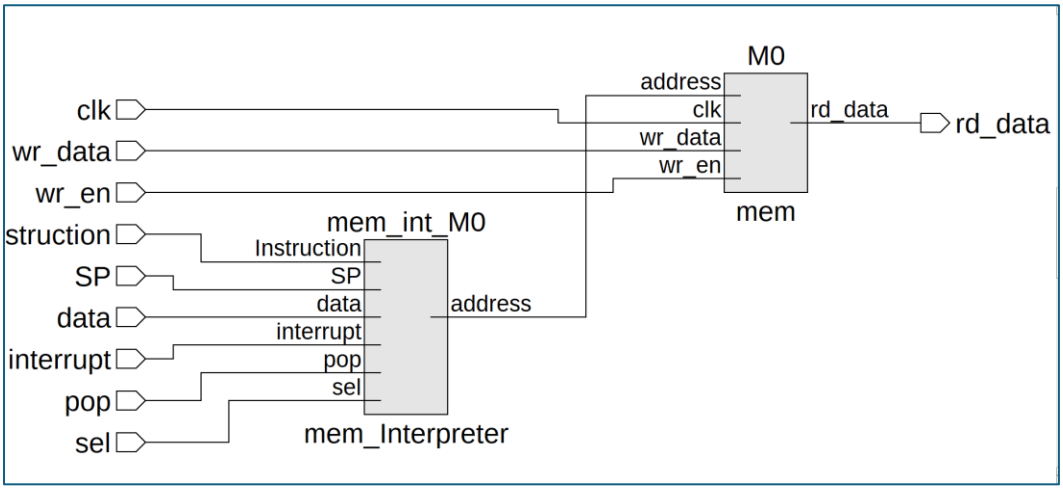

Figure 12: block diagram of the wrapped memory

5. Write Back Stage

In this stage, we write again in the register file so it use the same register file explained in the decode stage section but it gives the address to it to determine which register to write in and provides the register file with data needed to be written. We can fine MUXs on write data and write address port. The MUX in write data port to determine if the data written in the RF is from meory or ALU and the MUX in write address port to determine the address given from rb or ra.

6. Hazard Detection and Handling Unit

The design is based on comparing:

- Source registers of instructions in Decode and Execute stages
- Destination registers of instructions in Memory and Write-Back stages
- Register usage signals
- Write-enable signals

If a source register matches a destination register of a later stage that will write data, a hazard exists.

Forwarding Logic

Forwarding is used whenever data is available before register write-back:

- **Memory → Execute forwarding**
  Used when the Execute stage needs a register that the Memory stage will write.
- **Write-Back → Execute forwarding**
  Used when the needed data is only available in the Write-Back stage.
- **Write-Back → Decode forwarding**
  Used to prevent stalls during register read in the Decode stage.

This allows the pipeline to continue without stalling.

Stall Logic

Two types of stalls are generated:

- **Structural Stall**
  Issued when a memory instruction is active.
- **Data Stall (Load-Use Hazard)**
  Occurs when an Execute-stage instruction depends on data from a memory instruction that is not yet available. In this case, forwarding is insufficient, so a stall is required.

Use of Parallel assign Statements

The Hazard Unit is implemented as pure combinational logic using parallel assign statements because:

- All outputs depend only on current inputs
- There is no internal state or clock
- Each hazard signal is independent of the others
- All decisions are made within the same clock cycle

This ensures fast and deterministic hazard detection.

6.1 Hazard Detection and Handling Unit Testbench

This testbench applies to a set of directed test cases to verify the Hazard Unit's forwarding and stall logic. In the no-hazard case, all forwarding signals (fwd_*) and stall signals (stall_structural, stall_data) are expected to be deasserted. When a Decode stage source register matches the Write-Back destination, the corresponding Write-Back-to-Decode forwarding signal (fwd_A_wb_decode or fwd_B_wb_decode) is expected to assert while all stall signals remain low. When the Execute stage depends on a Write-Back result, the Write-Back-to-Execute forwarding signals (fwd_A_wb_execute or fwd_B_wb_execute) are expected to assert with no stalls. For Execute stage dependencies on the Memory stage, the Memory-to-Execute forwarding signals (fwd_A_memory_execute or fwd_B_memory_execute) are expected to assert. When a memory instruction is active without a data dependency, a structural stall is expected (stall_structural = 1). Finally, in a load-use data hazard where Execute requires a value produced by a memory instruction, forwarding is insufficient and a data stall is expected (stall_data = 1), while forwarding signals remain deasserted..

6.2 Hazard Detection and Handling Unit Results

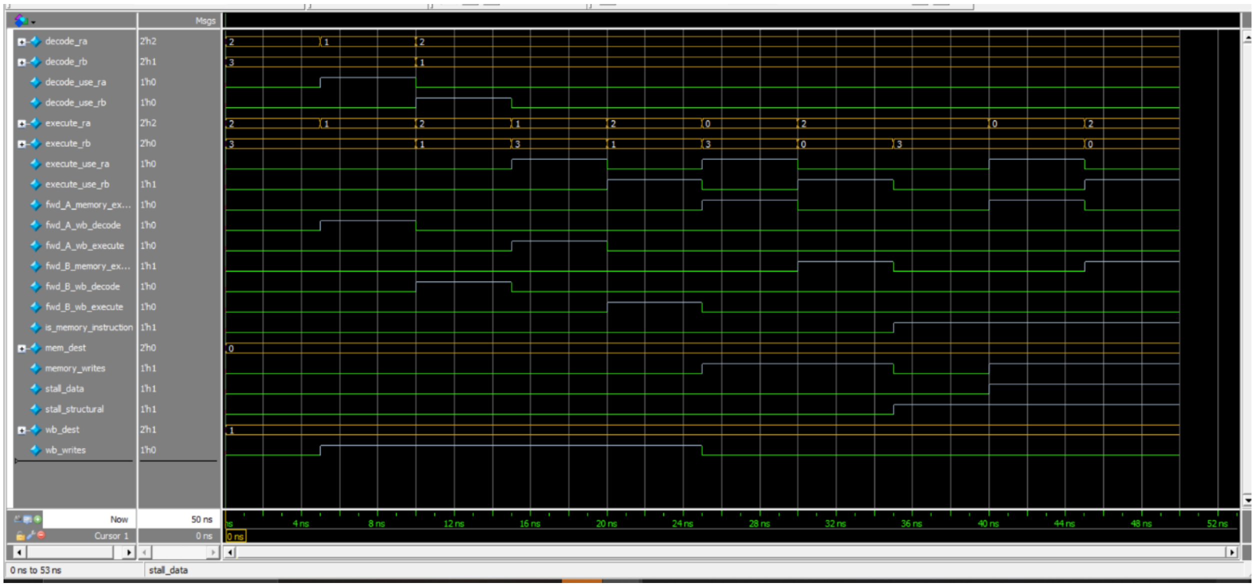

Figure 13: waveform of the hazard unit testbench

```
# Time | dec_use_ra dec_use_rb ex_use_ra ex_use_rb mem_wr wb_wr is_mem mem_dest wb_dest dec_ra dec_rb  ex_ra ex_rb | fwdA_mem fwdB_mem fwdAdec_wb fwdBdec_wb fwdAex_wb fwdBex_wb | stall_structural stall_data
# -----+--------------------------------------------------------------------------------------------+------------------------------------------
#                    5 |     0          0     0          0       0     0     0     00        01       10   11   10   11   |     0        0      0       0       0       0 |       0              0
#                   10 |     1          0     0          0       0     1     0     00        01       01   11   01   11   |     0        0      1       0       0       0 |       0              0
#                   15 |     0          1     0          0       0     1     0     00        01       10   01   10   01   |     0        0      0       1       0       0 |       0              0
#                   20 |     0          0     1          0       0     1     0     00        01       10   01   01   11   |     0        0      0       0       1       0 |       0              0
#                   25 |     0          0     0          1       0     1     0     00        01       10   01   10   01   |     0        0      0       0       0       1 |       0              0
#                   30 |     0          0     1          0       1     0     0     00        01       10   01   00   11   |     1        0      0       0       0       0 |       0              0
#                   35 |     0          0     0          1       1     0     0     00        01       10   01   10   00   |     0        1      0       0       0       0 |       0              0
#                   40 |     0          0     0          0       0     0     1     00        01       10   01   10   11   |     0        0      0       0       0       0 |       1              0
#                   45 |     0          0     1          0       1     0     1     00        01       10   01   00   11   |     1        0      0       0       0       0 |       1              1
#                   50 |     0          0     0          1       1     0     1     00        01       10   01   10   00   |     0        1      0       0       0       0 |       1              1
# Testbench finished.
```

Figure 14: the transcript to show the results of the testbench of the hazard unit

7. Top Module

It is the wrapper of the modules which instantiate every module then each module its input signal and output signal are handled using simple logic gates and MUXs to handle different scenarios and will be explained in the following sections:

7.1 PC

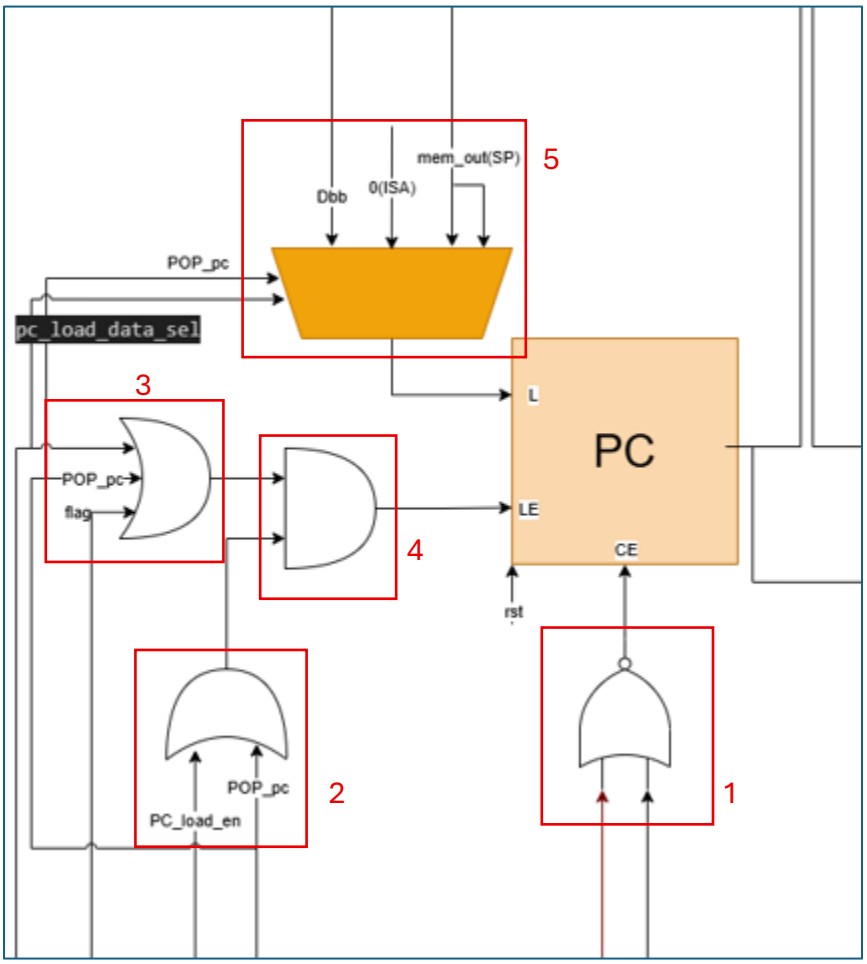

Figure 15:PC input and output selections

1.NOR Gate: The two inputs are stall_data (in case forced stall like load)(data hazard) and stall_structural (for any instruction that will use the memory like push)(structural hazard) coming directly from the Hazard_Unit where when any of them is high we need to stop the counter_en (the output of the gate) of the PC.

2.OR Gate: the two inputs which are pc_load_en which comes from the D/EX register to show that there is a probability to load the PC coming from all jumps (except of the return and load interrupt ones) as they jump in the EX stage. And the other input is pop_pc which comes from

EX/M register to show that there is a probability to load in the PC coming from the return jumps (loading PC saved in the stack).

3.OR Gate: The upper OR gate has three inputs which are flag to detect if the jump in the EX stage is taken and it's coming from the D/EX register, the second input is pop_pc to ensure the return functions has loaded the popped PC in the PC successfully and it comes from EX/M stage and the third input is pc_load_data_sel in case of loading interrupt it will be high and it comes from the D/EX stage to ensure when loading interrupt that the pc_load_en is high.

4. AND Gate: the two outputs of the OR gates are connected as an input to AND gate to ensure that both the conditions are applied when to load the PC.

5. 4x1 MUX: The selectors are pop_pc from EX/MEM register and pc_load_data_sel from D/EX register and the inputs of the mux is pop_pc,isa_address and the output of the memory, when pop_pc is high then pc_load_data_sel is don't care, the Load data is the output of the memory directly as it's the case of loading the pc coming from the stack, and when pop_pc is low then if the pc_load_data_sel is high the load data is the ISR address (which is constant 0) and if pc_load_data_sel is low the input is the D_b coming from D/EX register (if D_b needs forwarding it's forwarded as we will show in the forwarding muxes in EX stage).

7.2 Register file

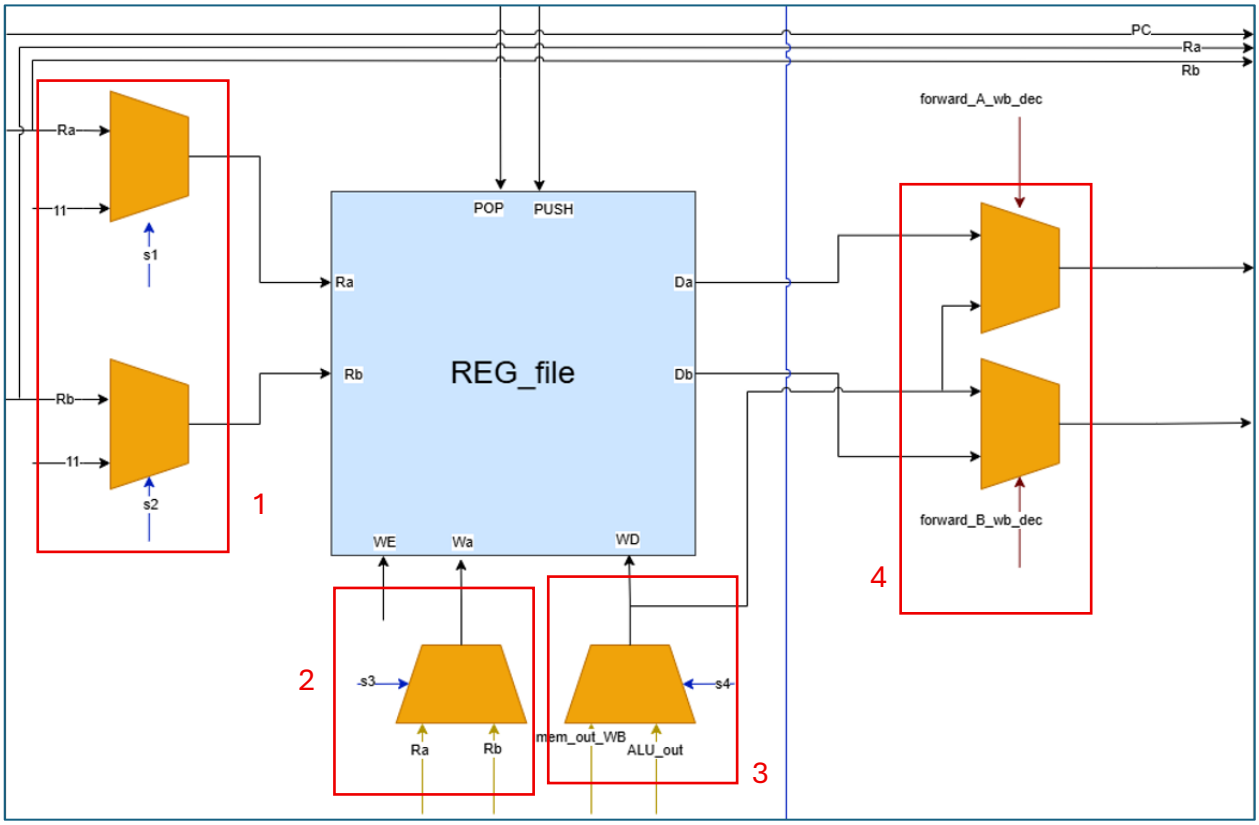

Figure 16: Register file input and output selections

1. Two Muxes to Ra and Rb: each has one selector(s1 and s2) each selector coming directly from the CU and the inputs are Ra(or Rb) from IF/D register and R3 address (11) (which is needed when push, pop and call), if the selector is high the R3 (SP) address is input to Ra or Rb and if the selector is low the Ra or Rb coming from the IF/D register is input to Ra or Rb.
2. Write address mux: It has selector and two inputs Ra and Rb all coming from the M/WB register, if the selector is high the Rb is input to write address of the reg (to choose which register) and if the selector is low Rb is input to write address of the reg.
3. Write data mux: It has selector and two inputs ALU output and memory output write back all coming from the M/WB register, if the selector is high the Mem_out is input in the write data(like in case of load) and if it's low the ALU_out (for any output not coming from memory) is input instead.
4. Two Muxes to forward Da or Db: each has one selector each coming directly from the Hazard_Unit_forward_A(orB)_wb_dec and has input of the output of Write data mux and data from the output of the register file(Da or Db),If the selector is high the forwarded data coming from the write data to the reg file is input as Da(or Db) to the register D/EX (in case of forwarding) and if the selector is low the normal Da(or Db) is input.

7.3 Execution stage block

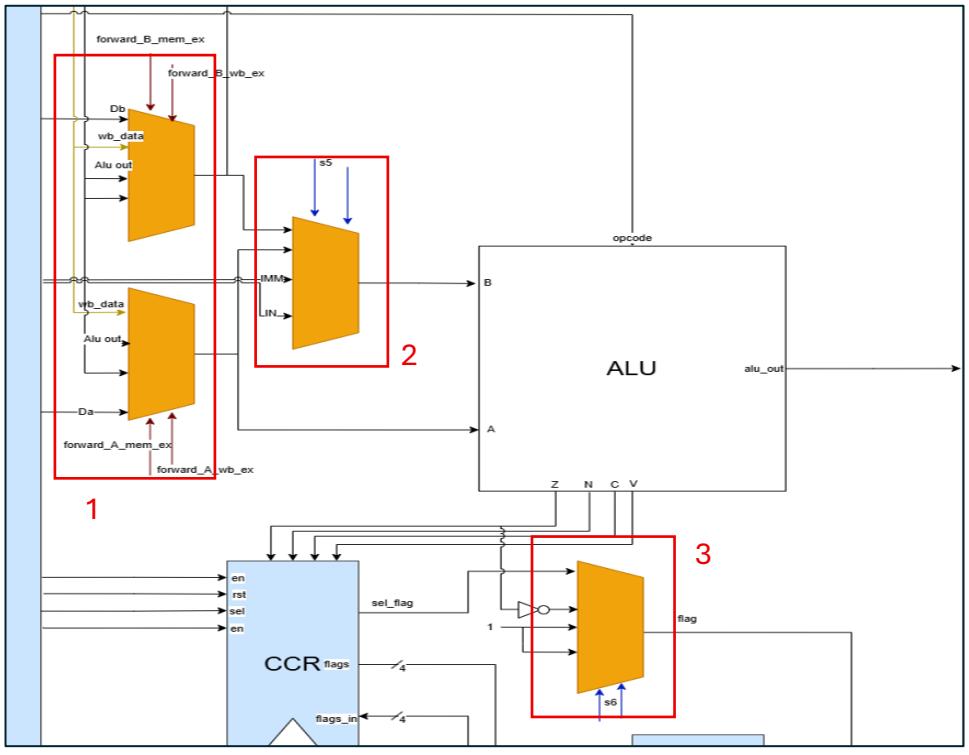

Figure 17:Excuation stage input and output selections

1. The two 4x1 MUXs: each has two selectors coming directly from the the Hazard_Unit which are forward_A_mem_ex and forward_A_wb_ex and has inputs wb_data, alu_out, and the output D/EX (ra or rb), if forward_A_mem_ex is high, and forward_A_wb_ex is don't care, and the data coming from the ALU_out from the EX/M register is input as the new Da(or Db), and if it's low, if the forward_A_wb_ex is high the wb_data coming from the write data in the reg file as discussed above, is input and if both selectors is low the normal Da coming from the D/EX register is input.

2. ALU_B Mux: It has 2_bit selector coming from the D/EX register and the inputs are Da, Db, immediate value and input value and when the selector value is:

- 0: pass the Da output from the mux above
- 1: pass the Db output from the mux above
- 2: pass the immediate value from the D/EX register
- 3: pass the input value from the D/EX register

3. 4x1 Flag Mux: It has 2_bit selector coming from the D/EX register:

- 0: pass the output of the CCR (for conditional jump operations)
- 1: pass the NOT of the zero-flag coming directly from the ALU (LOOP operation)
- 2 or 3: pass constant value 1 (unconditional jumps)

7.4 Memory wrapper

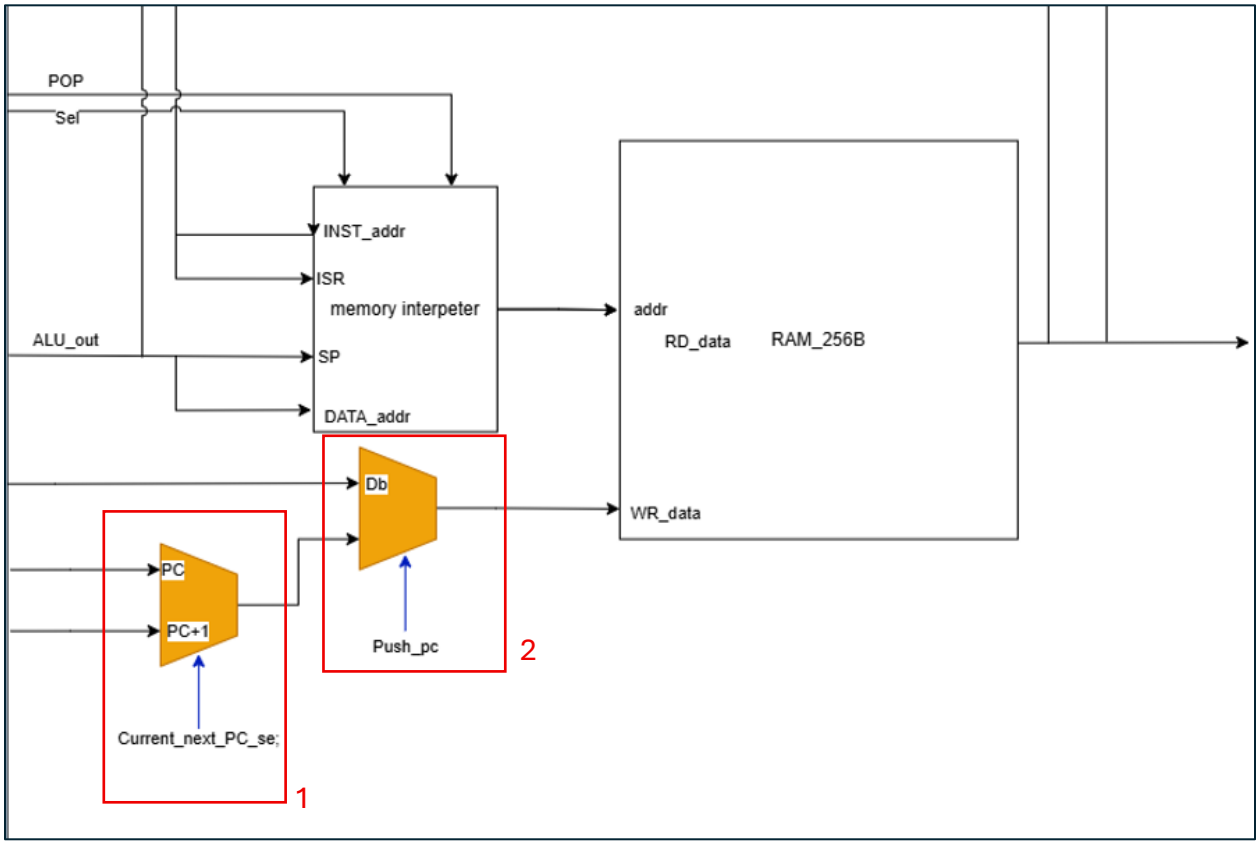

Figure 18: Memory input and output connections

1. Current next pc MUX: It has one selector and two inputs PC and PC + 1(to save PC when interrupt and PC + 1 in case of like call) all coming from EX/M register, the selector is current_next_pc_sel as if it's high the current PC will pass and if low the next PC (PC + 1) will pass.
2. Write data MUX: It has one selector and two inputs all coming from EX/M register, the selector is push_pc as if it's high the output of the mux above is input and if low the Db is input.

7.5 Output Logic

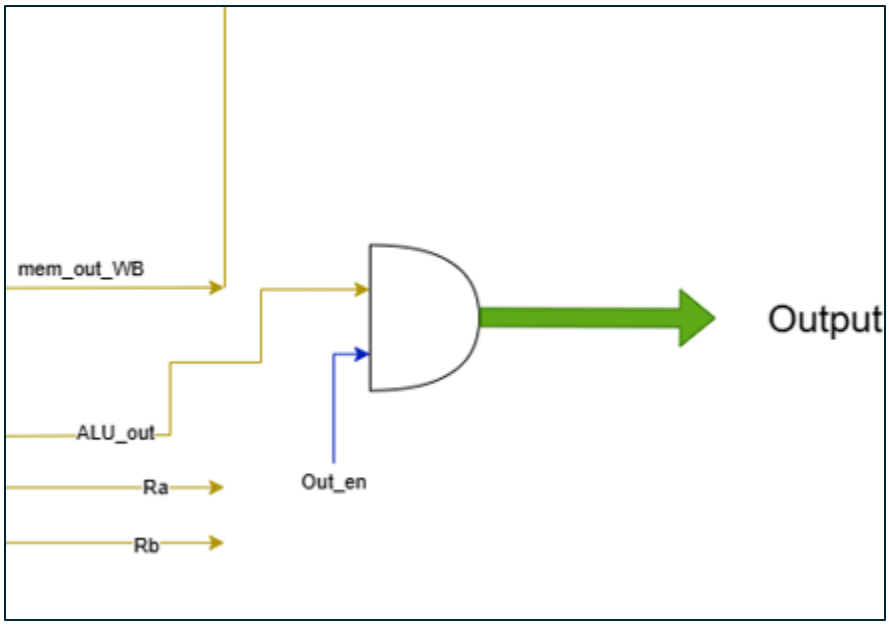

Figure 19: output logic schematic

Out AND for the output logic: If out_en coming from M/WB register is high the ALU_out coming from the M/WB register is input else 0 is input.

8.Processor Results

After testing the processor with 8 tests gathering most of the instructions including hazards and forwarding, and each test serves a certain testing purpose (The instructions in tests 1,2 and 3 only are separated by 5 clock cycles (filled with no op) to eliminate the dependencies between to test the single instructions). as follows:

- TEST 1: Load immediate, MOV, ADD, SUB, AND. OR. NOT, NEG, INC, DEC.
- TEST 2: STEC, RLC, CLRC, RRC, PUSH., POP, OUT, IN
- TEST 3: STD, LDD, STI, LDI
- TEST 4: JZ, JN
- TEST 5: JC, JV
- TEST 6: JMP, LOOP
- TEST 7: CALL, RTC
- TEST 8: INTRUPPT, RETURN FROM INTURRUPT

And the testbench code is:

```
module normal_tb ();
    reg clk, rst, interrupt_tb;
    reg [7:0] in_tb;
    wire [7:0] out_dut;

    // inist.
    top_module dut (clk, rst, interrupt_tb, in_tb, out_dut);

    initial begin
        $readmemb("test1.dat", dut.mem_wrapper_inst.M0.mem);

        clk = 0;
        forever begin
            #1 clk = ~clk;
        end
    end

    initial begin
        rst = 1;
        interrupt_tb = 0;
        in_tb = 8'hab;

        @(negedge clk);

        rst = 0;
```

```
        repeat(60) begin
            @(negedge clk);
        end

        $stop;
    end
endmodule
```

8.1 Test 1

8.1.1 Test 1 Assembly Code

```
LDM R0, 2           ; Load immediate 2 into R0
LDM R1, 7           ; Load immediate 7 into R1
MOV R2, R0          ; Copy R0 to R2 (R2 = 2)
ADD R0, R1          ; R0 = R0 + R1 = 2 + 7 = 9
SUB R0, R1          ; R0 = R0 - R1 = 9 - 7 = 2
AND R1, R2          ; R1 = R1 & R2 = 7 & 2 = 2
OR R2, R0           ; R2 = R2 | R0 = 2 | 2 = 2
NOT R0              ; R0 = ~R0 = ~2 = -3
NEG R1              ; R1 = -R1 (2's complement)
INC R2              ; R2 = R2 + 1 = 3
DEC R2              ; R2 = R2 - 1 = 2
```

8.1.2 Test 1 Results

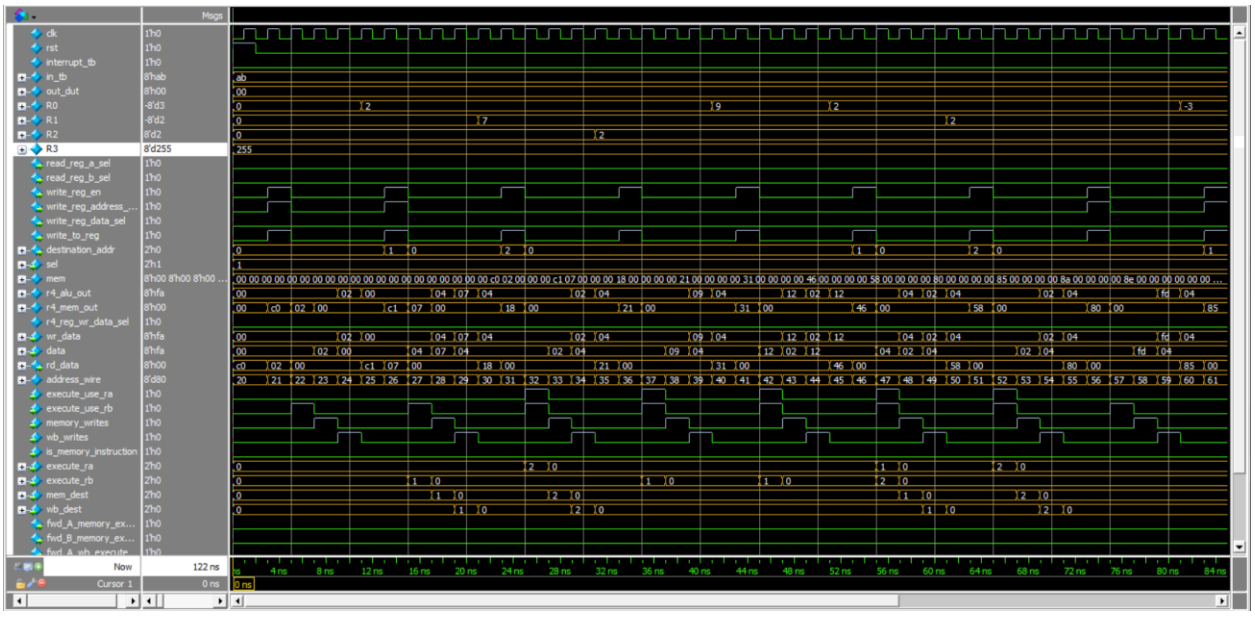

Figure 20: First part test 2 wavefrom

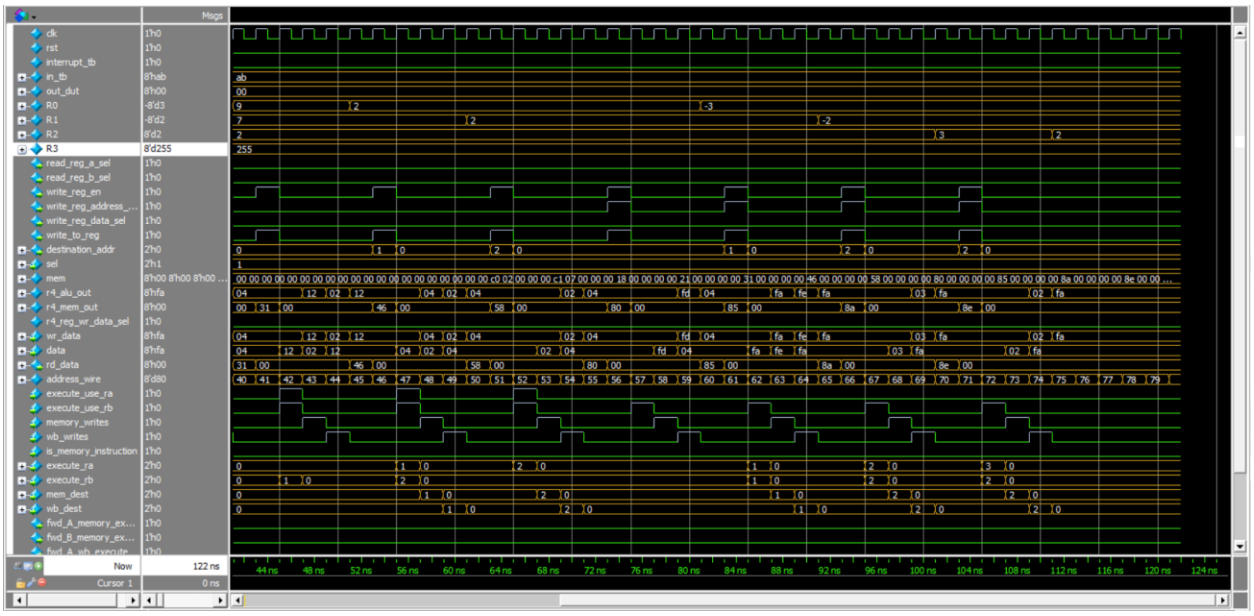

Figure 21: Second part test 2 wavefrom

And the output as expected the final output is = -3 in R0 and = -2 in R1 and = 2 R2 , as the wave started from address_line = 20 as the instructions part from 20-99 as in figure (20).

- The 1st instruction is done at clock 5 and R0 is loaded with 2.
- The 2nd instruction (address_line 25) is done at clock 10 and R1 is loaded with 7.
- The 3rd instruction (address_line 30) is done at clock 15 and R2 = 2 which moved from R0.
- The 4th instruction (address_line 35) is done at clock 20 and R0 = 9 which is R0 = 7 + R1 = 2.
- The 5th instruction (address_line 40) is done at clock 25 and R0 = 2 which is R0 = 9 - R1 = 7.
- The 6nd instruction (address_line 45) is done at clock 30 and R1 = 2 which is R1 = 7 & R2 = 2
- The 7th instruction (address_line 50) is done at clock 35 and R2 = 2 which is R1 = 2 | R2 = 2.
- The 8th instruction (address_line 55) is done at clock 40 and R0 = - 3 which is !(R0).
- The 9th instruction (address_line 60) is done at clock 45 and R1 = - 2 which is (2's Complement of R1).
- The 10th instruction (address_line 60) is done at clock 50 and R2 = 3 which is R2 + 1.
- The 11st instruction (address_line 60) is done at clock 55 and R2 = 2 which is R2 - 1.

8.2.1 Test 2

8.2.1 Test 2 Assembly code

```
LDM R0, 2           ; Load immediate 2 into R0
LDM R1, 7           ; Load immediate 7 into R1
MOV R2, R0          ; Copy R0 to R2 (R2 = 2)
SETC                ; Set carry flag C = 1
RLC R0              ; Rotate R0 left through carry (R0 = 5)
CLRC                ; Clear carry flag C = 0
RRC R1              ; Rotate R1 right through carry (R1 = 3)
PUSH R2             ; Push R2 onto stack
POP R1              ; Pop from stack into R1(R1 = 2)
OUT R1              ; Output R1 to output port (Out port = 2)
IN R0               ; Read from input port into R0 (R0 = 0)
```

8.2.2 Test 2 Results

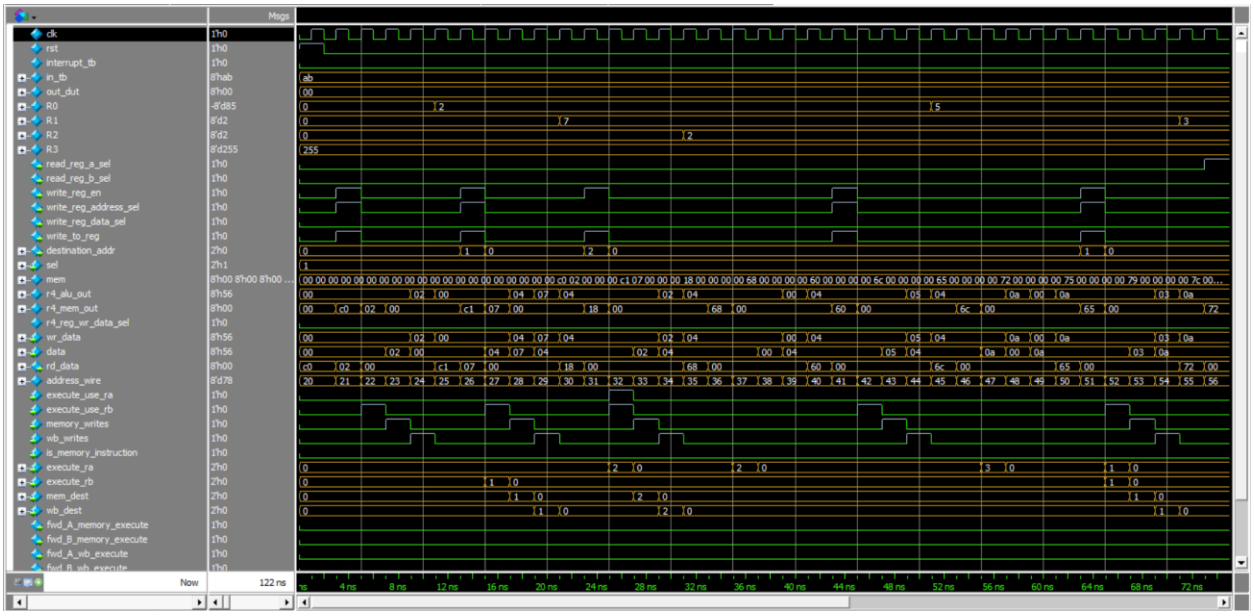

Figure 22: First Part of the waveform of test 2

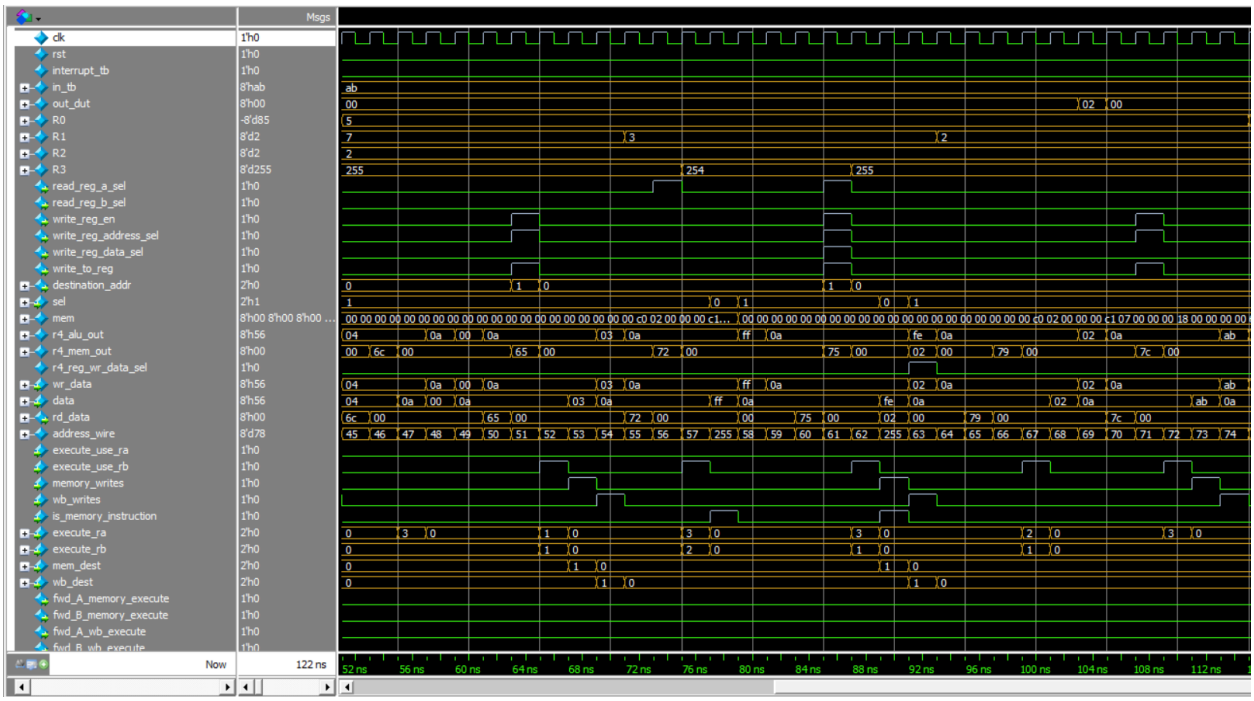

Figure 23: Second Part Test 2 Wavefrom

- The 1st instruction is done at clock 5 and R0 is loaded with 2.
- The 2nd instruction (address_line 25) is done at clock 10 and R1 is loaded with 7.
- The 3rd instruction (address_line 30) is done at clock 15 and R2 = 2 which is moved from R0.
- The 4th instruction (address_line 35) is done at clock 20 and C is set with = 1.
- The 5th instruction (address_line 40) is done at clock 25 and R0 = 5 which R0 is rotated left.
- The 6nd instruction (address_line 45) is done at clock 30 and C is set with = 0.
- The 7th instruction (address_line 50) is done at clock 35 and R1 = 3 which R1 is rotated right.
- The 8th instruction (address_line 55) is done at clock 40 and R3 = 254 which is push R2 = 2.
- The 9th instruction (address_line 60) is done at clock 45 and R3 = 255 and R1 = 2 which is pop from stack.
- The 10th instruction (address_line 60) is done at clock 50 and output port = 2 as the R1 = 2 is output.
- The 11st instruction (address_line 60) is done at clock 55 and R0 = 0 as input port = 0.

8.3 Test 3

8.3.1 Test 3 Assembly Code

```
LDM R0, 2           ; Load immediate 2 into R0
LDM R1, 7           ; Load immediate 7 into R1
MOV R2, R0          ; Copy R0 to R2 (R2 = 2)
STD R1, 3           ; Store R1 to memory [3]
LDD R2, 3           ; Load from memory[3] into R2 (R2 = 7) (to test the STD)
STI R0, R1          ; Store R1 to memory (SAME) R1 mem[R0]
LDI R0, R0          ; Load from memory[R0] into R0 (R0 =7)
```

8.3.2 Test 3 Results

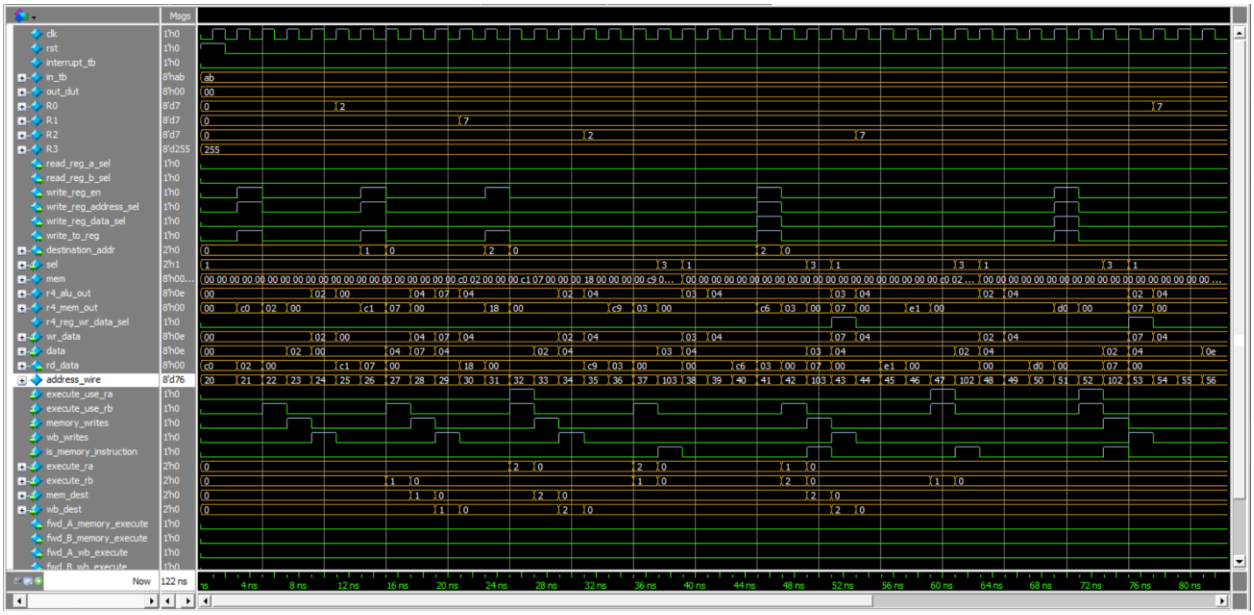

Figure 24: Test 3 Wavefrom

- The 1st instruction is done at clock 5 and R0 is loaded with 2.
- The 2nd instruction (address_line 25) is done at clock 10 and R1 is loaded with 7.
- The 3rd instruction (address_line 30) is done at clock 15 and R2 = 2 which is moved from R0.
- The 4th instruction (address_line 35) is done at clock 20 which is store R1 into memory 3 + offset = 100.
- The 5th instruction (address_line 40) is done at clock 25 and R2 = 7 which is load R2 from memory of memory 3 + offset = 100.

- The 6nd instruction (address_line 45) is done at clock 30 and store the value R1 to memory of address R0.
- The 7th instruction (address_line 50) is done at clock 35 and R0 = 7 which is load from memory of address R0 into R0.

8.4 Test 4

The purpose of this test is to test the conditional jump if zero and jump if negative instructions.

8.4.1 Test 4 Assembly Code

```
LDM R0, 3            ; R0 = 3
LDM R1, 7            ; R1 = 7
ADD R0, R1           ; R0 = 10 (Z=0, N=0)
JZ R0                ; Jump to R0 if Z = 1 (Not taken)
SUB R1, R1           ; R1 = 0 (Z=1)
JZ R0                ; Jump to R0 if Z=1 (taken if R0 has valid address)
OUT R3               ; Output SP (this should be skipped).
INC R0               ; R0++ (Skipped)
LDM R2, 32           ; R2 = 32
SUB R2, R1           ; R2 = 32 - 0 = 32 (Z=0, N=0)
JN R2                ; Jump to R2 if N=1 (not taken)
SUB R1, R0           ; R1 = 0 - R0 (negative)
JN R2                ; Jump to R2 if N=1 (taken and go to address 32+20 = 52)
INC R0               ; R0++(this line at address 52)
```

8.4.2 Test 4 Results

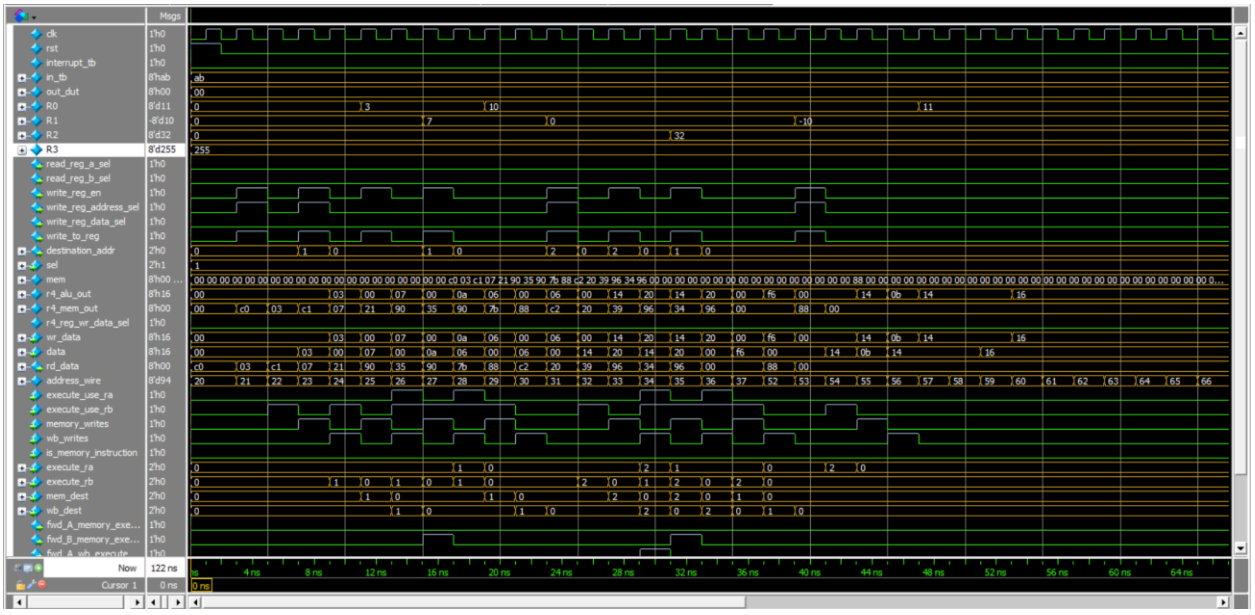

Figure 25: Test 4 Wavefrom

The rest of explanation of waveforms will be found in the demo video.

8.5 Test 5

The purpose of this test is to test the conditional jump if carry, jump if zero and jump if overflow instructions.

8.5.1 Test 5 Assembly Code

```
LDM R0, 2           ; R0 = 2
LDM R1, 15          ; R1 = 15
JC R1               ; Jump to R1 if C=1 (not taken)
SETC                ; Set carry flag
JC R1               ; Jump to R1 if C=1 (taken)(15+20 = 35)
JZ R0               ; Jump to R0 if Z=1(skipped)
OUT R3              ; Output SP(skipped)
INC R0              ; R0++(skipped)
LDM R2, 32          ; R2 = 32(skipped)
SUB R2, R1          ; R2 = 32 - 15 = 17(skipped)
JN R2               ; Jump to R2 if N=1(skipped)
NOP                 ;    (land here)
INC R0              ; R0++ (R0 = 3)
LDM R0, 32          ; R0 = 32
LDM R2, 127         ; R2 = 127 (max positive)
INC R2              ; R2 = -128 (overflow! V=1)
JV R0               ; Jump to R0 (32+20 = 52)
DEC R0              ; R0—
```

8.5.2 Test 5 Results

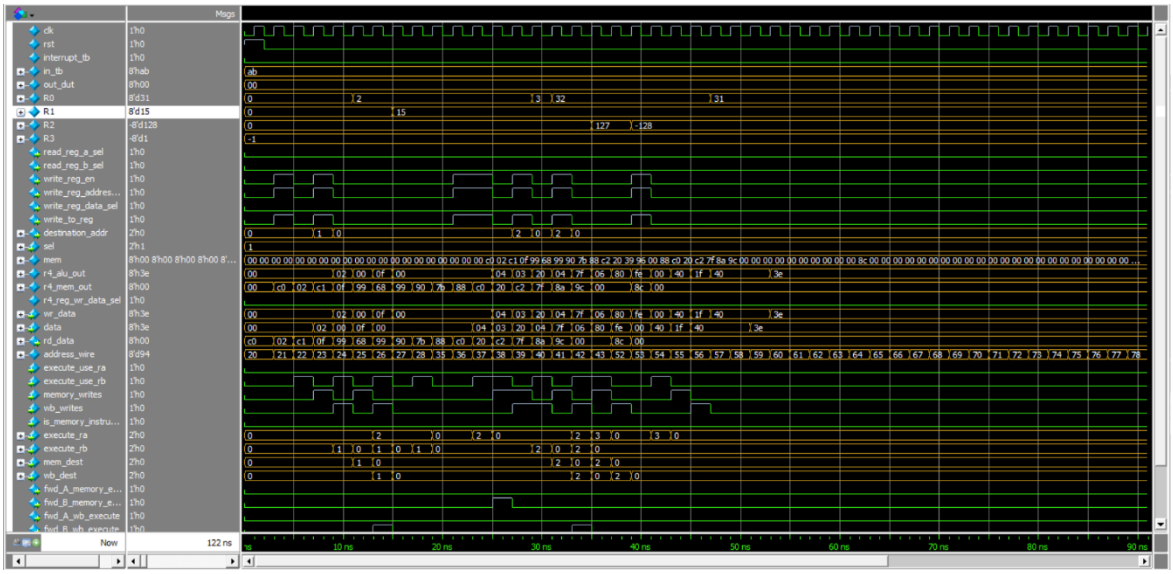

Figure 26: Test 5 Wavefrom

8.6 Test 6

The purpose of this test is to test the unconditional jump instructions.

8.6.1 Test 6 Assembly Code

```
LDM R0, #3          ; R0 = 3 (loop counter)
LDM R1, #15         ; R1 = 15
JMP R1              ; Unconditional jump to address in R1 (35)
SETC                ; Set carry (skipped by JMP)
JC R1               ; Jump if carry (skipped by JMP)
JZ R0               ; Jump if zero (skipped by JMP)
OUT R3              ; Output SP (skipped by JMP)
INC R0              ; R0++ (skipped by JMP)
LDM R2, #32         ; R2 = 32 (skipped by JMP)
SUB R2, R1          ; R2 = 32 - 15 (skipped by JMP)
JN R2               ; Jump if negative (skipped by JMP)
INC R2              ; R2++ (R2 = 1)
LOOP R0, R1         ; Decrement R0, jump to R1 if R0 != 0
DEC R2              ; R2—
```

8.6.2 Test 6 Results

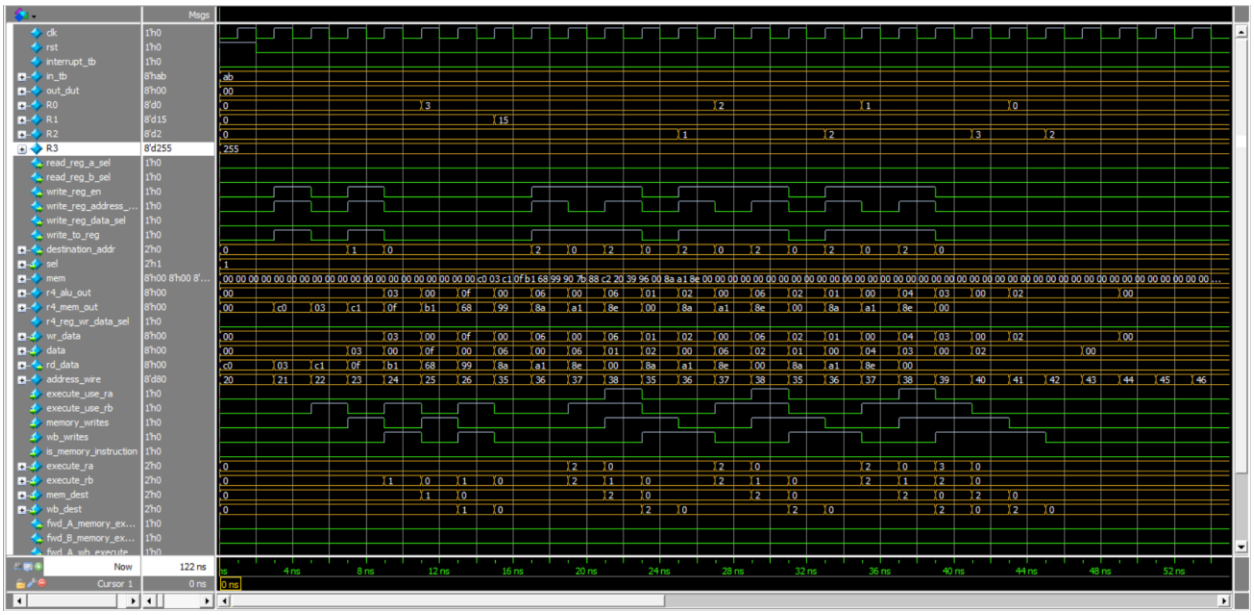

Figure 27: Test 6 Wavefrom

8.7 Test 7

The purpose of this test is to test the call and return functions.

8.7.1 Test 7 Assembly Code

Main Program:

```
LDM R0, #3          ; R0 = 3
LDM R1, #15         ; R1 = 15
CALL R1             ; Call subroutine at address R1 (push PC, jump) (mr shes you
will notice that R3 which is Stack pointer will be decreased ha.)
DEC R2              ; After return : R2--
DEC R2              ; R2--

 CALL -  Subroutine
MOV R2, R0          ; R2 = R0    (R2 = 3)
ADD R0, R2          ; R0 = R0 + R2 (R 0 = 6)
RTI                 ; Return from interrupt/subroutine (pop PC)
INC R2              ; ( this should not be executed).
```

8.7.2 Test 7 Results

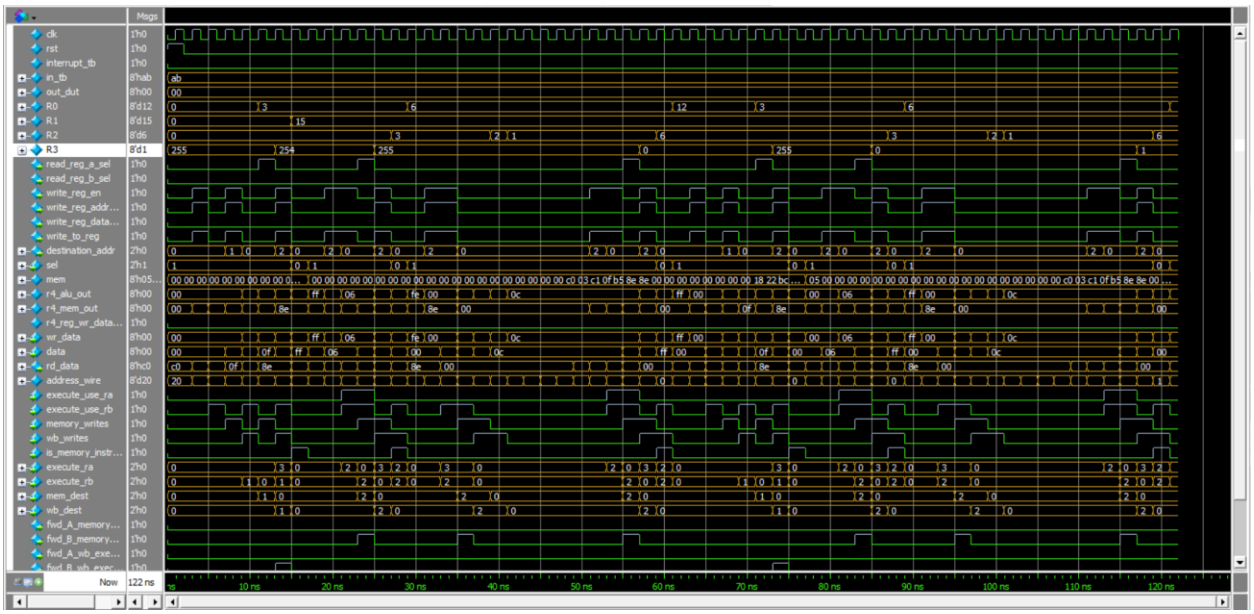

Figure 28: Test 7 Wavefrom

8.8 Test 8

The purpose of this test is to test the interrupt, return from interrupt and stall due to data hazard. Note: this test has a different testbench and it is:

```
module interrupt_tb ();
    reg clk, rst, interrupt_tb;
    reg [7:0] in_tb;
    wire [7:0] out_dut;

    // inist.
    top_module dut (clk, rst, interrupt_tb, in_tb, out_dut);

    initial begin
        $readmemb("test8.dat", dut.mem_wrapper_inst.M0.mem);

        clk = 0;
        forever begin
            #1 clk = ~clk;
        end
    end

    initial begin
        rst = 1;
        interrupt_tb = 0;
        in_tb = 8'hab;

        @(negedge clk);

        rst = 0;

        repeat(5) begin
            @(negedge clk);
        end

        interrupt_tb = 1;

        @(negedge clk);

        interrupt_tb = 0;

        repeat(50) begin
            @(negedge clk);
        end

        $stop;
    end
endmodule
```

8.8.1 Test 8 Assembly Code

```
OUT R0              ; Output R0
OUT R1              ; Output R1
OUT R2              ; Output R2
OUT R3              ; Output R3
SETC                ; Set carry flag (here we want to make sure that the flag s
                    ;from original program is stored in the status register)
RTI                 ; Return from interrupt

Main Program starts at address 0x14:
LDM R0, 3           ; R0 = 3
LDM R1, 15          ; R1 = 15
SUB R2, R2          ; R2 = 0
(warning interrupt) jump to interrupt section.
INC R2              ; R2 = 1  (here we return from interrupt, and you should
                    ;notice the flags are back to normal)
INC R2              ; R2 = 2
INC R2              ; R2 = 3
INC R2              ; R2 = 4
INC R2              ; R2 = 5
LDI R2, R0          ;this load and move are made to test RAW hazard (notice stall)
MOV R1, R0          ; R1 = R0
```

8.8.2 Test 8 Results

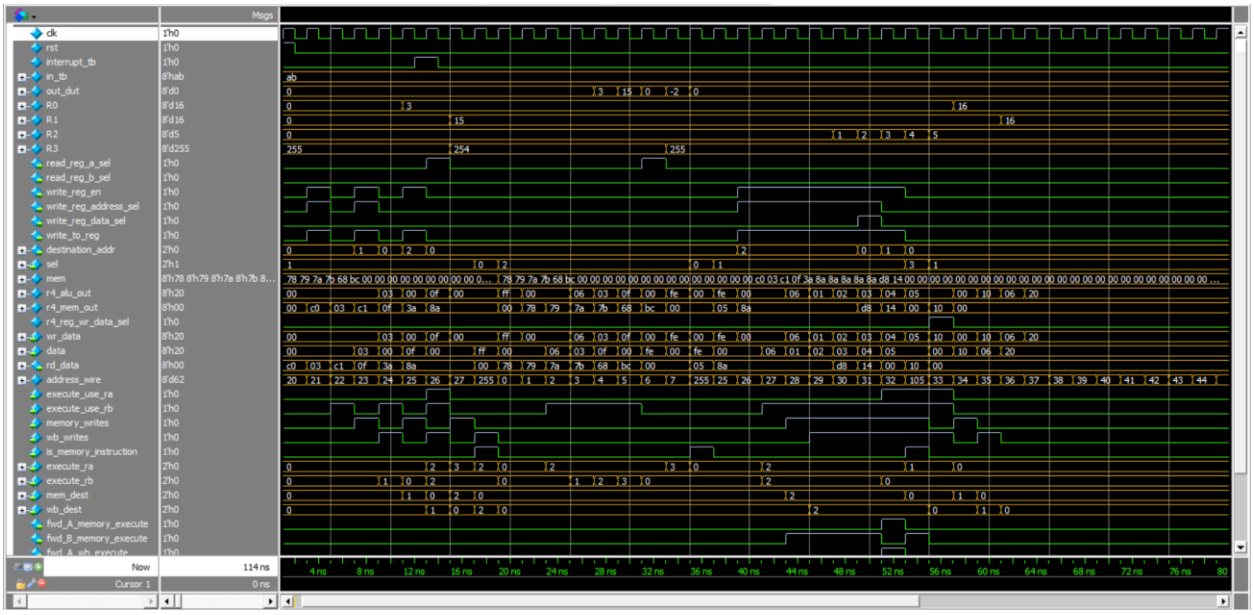

Figure 29: Test 8 Wavefrom

9.Utilization of processor

In this part, we put all the modules on Vavido to be utilized, and we added to it a constraint file to see what the processor will work on without any violations in hold and setup times. We used FBGA board named “XC7A35TICPG236-1L” to implement on it.

9.1 The schematic after synthesis:

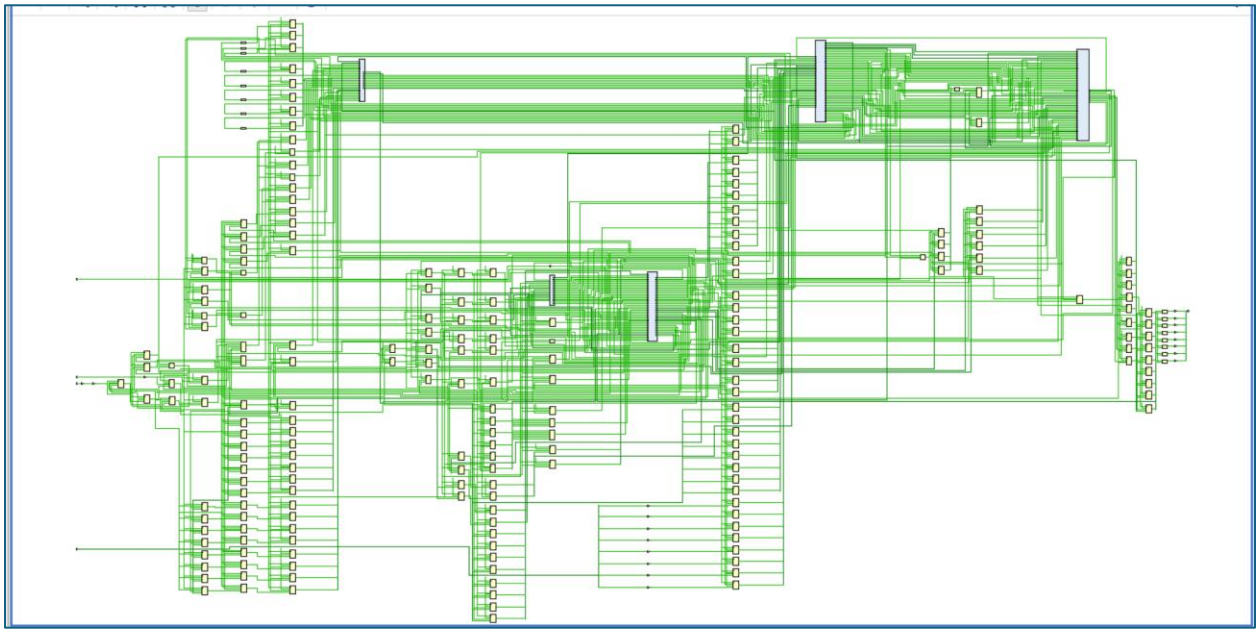

Figure 30: schematic of the processor after synthesizing

9.2 The utilization report of used blocks in the synthesizing:

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Bonded IOB (106)	BUFGCTRL (32)
top_module	337	272	16	8	19	1
CU_inst (CU)	85	33	0	0	0	0
EX_stage_inst (EX_sta...	59	8	0	0	0	0
mem_wrapper_inst (m...	58	0	16	8	0	0
PC_inst (PC)	75	8	0	0	0	0
reg_file_inst (reg_file)	35	32	0	0	0	0

Figure 31: utilization report

Resource	Utilization	Available	Utilization %
LUT	337	20800	1.62
LUTRAM	32	9600	0.33
FF	272	41600	0.65
IO	19	106	17.92

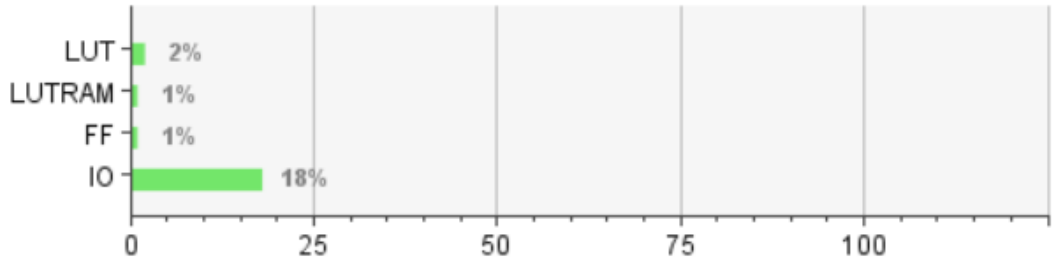

Figure 32: summary of used blocks

9.3 frequency of the clock

We added a constraint file that has period of 11ns to test the timing analysis, and we found it to pass all without any violation for hold time or setup time.

9.3.1 The frequency table

Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk_pin	{0.000 5.000}	11.000	90.909

Figure 33: the clock frequency

9.3.2 The STA

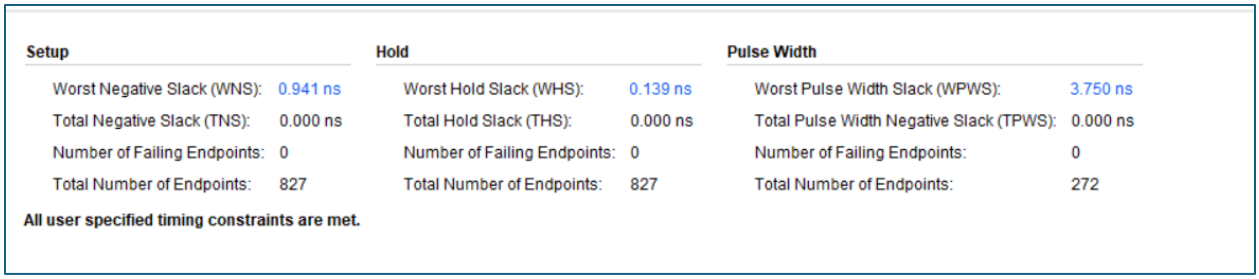

Figure 34: the STA from the tool

10. Assembler

We made an assembler that converts the assembly code to binary code and can be loaded to the RAM to run a program how to generate the file that has the opcodes there is a guide on how to generate the file. It has some restrictions like when add the code either in interrupt part or main part the code should have a tab space before it and the numbers are written in either decimal or hexadecimal but to make it hexadecimal it should be written in format “0xnumber”. It also had the ability to write a comment but should has semi-colon “;” before it.

Example for a code:

```
interrupt_start:
    ADD R0, R2
    SUB R2, R1
    INC R0
    RTI
main_start:
    LDM R0, 2           ; Load immediate 2 into R0
    LDM R1, 7           ; Load immediate 7 into R1
    MOV R2, R0          ; Copy R0 to R2 (R2 = 2)
    ADD R0, R1          ; R0 = R0 + R1 = 2 + 7 = 9
    SUB R0, R1          ; R0 = R0 - R1 = 9 - 7 = 2
    AND R1, R2          ; R1 = R1 & R2 = 7 & 2 = 2
    OR R2, R0           ; R2 = R2 | R0 = 2 | 2 = 2
    NOT R0              ; R0 = ~R0 = ~2 = 0xFD
    NEG R1              ; R1 = -R1 (2's complement)
    STD R1, 3          ; store value in R1 to the address 3 in memory
    INC R2              ; R2 = R2 + 1 = 3
    DEC R2              ; R2 = R2 - 1 = 2
```